BUSINESS ANALYTICS PROJECT

RETAIL ORDERS BUSINESS ANALYTICS PROJECT

Student Name: Vijay Bisht

Roll Number: T004

Class: TY B.Sc. IT

Division: A

Subject: Applied Business Analytics Practical

Academic Year: 2026–27

College Name: SHETH L.U.J. and Sir M.V.College of Arts ,Science and Commerce

Department Name: BscIT

Faculty Name: Chetna Tanavade

TABLE OF CONTENTS

1. Title Page
2. Table of Contents
3. Introduction
4. Problem Statement
5. Objectives
6. Business Scenario / Domain Description
7. Dataset Description
8. Data Collection
9. Data Preprocessing
10. Exploratory Data Analysis
11. Descriptive Statistical Analysis
12. Distribution Analysis
13. Correlation Analysis
14. Latitude and Longitude Analysis
15. Business Intelligence Dashboard
16. Probability and Probability Distributions
17. Decision Making Under Uncertainty
18. Sampling and Sampling Distribution
19. Confidence Interval Estimation
20. Bootstrapping
21. Hypothesis Testing
22. A/B Testing
23. Time Series Analysis
24. Forecast Accuracy
25. Optimization Modeling
26. Simulation Modeling
27. ANOVA
28. Experimental Design
29. Factorial Experiment
30. Business Insights and Findings
31. Business Recommendations
32. Conclusion
33. Tools and Technologies
34. Appendix – Complete Python Code

1. INTRODUCTION

Business Analytics is the process of using data, statistical methods, visualization, and analytical techniques to support business decision-making.

The Retail Orders Business Analytics Project focuses on analyzing delivery-related data to understand business performance, delivery operations, geographical locations, and useful patterns in the dataset. The analysis helps identify important information that can support better business decisions and improve operational planning.

Python is used to perform data preprocessing, Exploratory Data Analysis (EDA), descriptive statistical analysis, distribution analysis, correlation analysis, and data visualization. Latitude and longitude analysis is also performed to visualize the geographical distribution of delivery locations.

Advanced Business Analytics techniques, including hypothesis testing, A/B testing, optimization, simulation, and experimental design, can be demonstrated using clearly labeled illustrative business scenarios where the original dataset does not contain the required experimental information.

The main aim of this project is to apply Business Analytics techniques to The Retail Orders dataset and demonstrate how data-driven analysis can help understand delivery operations and support effective business planning.

2. PROBLEM STATEMENT

Retail Orders is a logistics and delivery platform that handles delivery operations across different locations. Analyzing delivery-related data is important to understand operational patterns, geographical distribution, and business performance.

The main problem is to analyze the available Retail Orders dataset and extract useful information that can help improve delivery planning and business decision-making. By using Business Analytics techniques, the project aims to identify patterns in the available data and present the findings through statistical analysis and visualizations.

The analysis focuses on the following areas:

- Understanding the structure and characteristics of The Retail Orders dataset
- Identifying missing values and duplicate records.
- Performing Exploratory Data Analysis (EDA).
- Calculating descriptive statistics for numerical variables.
- Studying distributions and relationships between variables.
- Analyzing latitude and longitude to visualize delivery locations.
- Presenting important patterns through charts and visualizations.
- Demonstrating advanced analytical techniques using illustrative business scenarios where required.
- Providing useful business insights and recommendations based on the analysis.

3. OBJECTIVES

The main objectives of this Applied Business Analytics project are:

1. To understand and analyze The Retail Orders dataset
2. To perform data preprocessing and check missing values and duplicate records.
3. To perform Exploratory Data Analysis (EDA) to understand the dataset.
4. To calculate descriptive statistics for numerical variables.
5. To study the distribution of data using statistical methods and charts.
6. To identify relationships between numerical variables through correlation analysis.
7. To analyze latitude and longitude data to understand the geographical distribution of delivery locations.
8. To create visualizations for better understanding of business data.
9. To apply probability and probability distributions to business scenarios.
10. To demonstrate decision-making under uncertainty.
11. To perform sampling, confidence interval estimation, and bootstrapping.
12. To conduct hypothesis testing for statistical analysis.
13. To demonstrate A/B testing using an illustrative business scenario.
14. To study time series analysis and forecasting where suitable data is available.
15. To demonstrate optimization and simulation techniques using proposed business scenarios.
16. To perform ANOVA and experimental design where applicable.
17. To demonstrate factorial experiments using illustrative data.
18. To identify useful business insights from the analysis.
19. To provide business recommendations based on analytical findings.
20. To understand how Business Analytics supports effective planning and decision-making in delivery operations.

4. BUSINESS SCENARIO / DOMAIN DESCRIPTION

Domain: Retail / E-Commerce

The project represents a logistics and delivery business that provides transportation and delivery services to customers. The business contains information about:

· Delivery locations
· Latitude and Longitude
· Delivery operations
· Geographical distribution
· Customer locations

· Delivery patterns

Business analytics can help Retail Orders understand delivery patterns, geographical distribution, and operational performance to support better planning and decision-making.

5. DATASET DESCRIPTION

The dataset used in this project is: Retail_Orders_Analytics_Dataset.csv

The dataset contains **500 records and** 16 columns .

Important attributes :

Column	Description
Order_ID	Unique order identifier
Order_Date	Order date
City	City
State	State
Region	Geographic region
Latitude	Latitude coordinate
Longitude	Longitude coordinate
Product	Delivery service/product
Category	Service category
Quantity	Quantity
Distance_KM	Delivery distance
Sales	Sales amount
Profit	Profit amount
Payment_Mode	Payment method
Customer_Type	Business or Individual
Delivery_Status	Delivered or Cancelled

6. DATA COLLECTION

The dataset was prepared for the project in CSV format.

The file used in the project is: Retail_Orders_Analytics_Dataset.csv

The dataset contains retail transaction information required to perform different Business Analytics techniques.

The CSV file was imported into Python using the Pandas library.

Code:

```
df = pd.read_csv("Retail_Orders_Analytics_Dataset.csv")
```

Output:

```
DATA LOADED SUCCESSFULLY
Number of rows: 500
Number of columns: 16
```

7. DATA PREPROCESSING

Data preprocessing was performed before analysis to improve the quality and consistency of The Retail Orders dataset.
Steps performed:

1. Loaded the CSV dataset.
2. Checked dataset dimensions.
3. Checked column names.
4. Checked missing values.
5. Checked duplicate records.
6. Converted Order_Date into datetime format.
7. Created Order Month from Order Date.
8. Checked numerical and categorical data types.
9. Identified duplicate records.
10. Checked the consistency of important fields such as Sales, Profit, Quantity, and Distance_KM.

Important formula
Order Month: Order Month is extracted from Order_Date to support monthly analysis.
Screenshot: Insert the output of preprocessing, missing-value check and duplicate check.

8. EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis was performed to understand the structure, characteristics and patterns present in the Retail Orders delivery dataset .

Code:

```
import pandas as pd

df = pd.read_csv("Retail_Orders_Analytics_Dataset.csv")

# 12.1 First five records
print("12.1 FIRST FIVE RECORDS")
print(df.head())
```

```
# 12.2 Last five records
print("\n12.2 LAST FIVE RECORDS")
print(df.tail())

# 12.3 Dataset shape
print("\n12.3 DATASET SHAPE")
print(df.shape)

# 12.4 Column names
print("\n12.4 COLUMN NAMES")
print(df.columns.tolist())

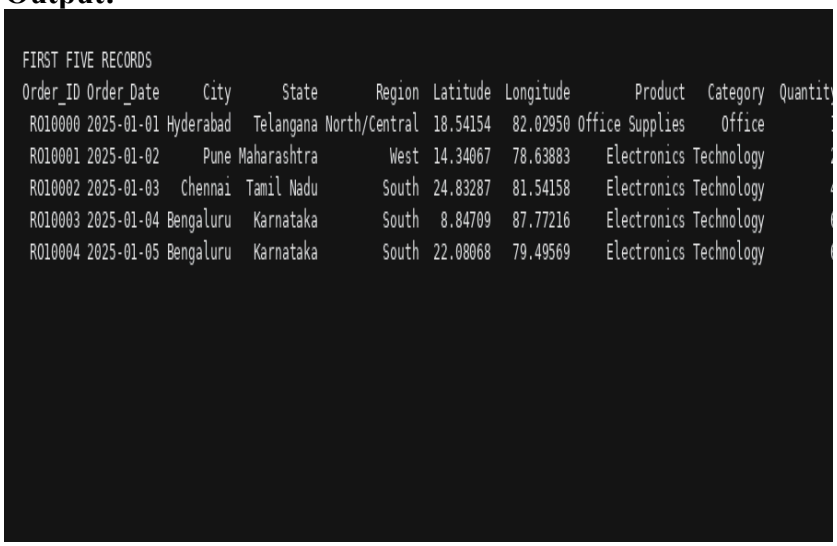
# 12.5 Data types
print("\n12.5 DATA TYPES")
print(df.dtypes)

# 12.6 Missing values
print("\n12.6 MISSING VALUES")
print(df.isnull().sum())

# 12.7 Duplicate records
print("\n12.7 DUPLICATE RECORDS")
print(df.duplicated().sum())

# 12.8 Unique values
print("\n12.8 UNIQUE VALUES")
 print(df.nunique())
```

Output:



```
FIRST FIVE RECORDS
Order_ID Order_Date     City      State       Region  Latitude  Longitude        Product   Category  Quantity
 RO10000 2025-01-01 Hyderabad   Telangana North/Central  18.54154   82.02950 Office Supplies    Office         1
 RO10001 2025-01-02      Pune Maharashtra         West  14.34067   78.63883    Electronics Technology         2
 RO10002 2025-01-03   Chennai  Tamil Nadu        South  24.83287   81.54158    Electronics Technology         4
 RO10003 2025-01-04 Bengaluru   Karnataka        South   8.84709   87.77216    Electronics Technology         6
 RO10004 2025-01-05 Bengaluru   Karnataka        South  22.08068   79.49569    Electronics Technology         6
```

LAST FIVE RECORDS

Order_ID	Order_Date	City	State	Region	Latitude	Longitude	Product	Category	Quan
R010495	2026-06-25	Delhi	Delhi	North/Central	20.87616	86.66232	Office Supplies	Office	
R010496	2026-06-26	Delhi	Delhi	North/Central	18.73916	74.92143	Furniture	Home & Furniture	
R010497	2026-06-27	Hyderabad	Telangana	North/Central	16.70170	87.83208	Accessories	Office	
R010498	2026-06-28	Bengaluru	Karnataka	South	17.90978	75.66506	Office Supplies	Office	
R010499	2026-06-30	Hyderabad	Telangana	North/Central	16.17379	85.86829	Accessories	Office	

12.3 DATASET SHAPE
(500, 16)

12.4 COLUMN NAMES
['Order_ID','Order_Date','City','State','Region','Latitude','Longitude','Product','Category','Quantity','Distance_KM','Sales'

12.5 DATA TYPES
Order_ID object | Order_Date datetime64[ns] | City object
State object | Region object | Latitude float64 | Longitude float64
Product object | Category object | Quantity int64 | Distance_KM float64
Sales float64 | Profit float64 | Payment_Mode object | Customer_Type object
Delivery_Status object

MISSING VALUES

Order_ID	0
Order_Date	0
City	0
State	0
Region	0
Latitude	0
Longitude	0
Product	0
Category	0
Quantity	0
Distance_KM	0
Sales	0
Profit	0
Payment_Mode	0
Customer_Type	0
Delivery_Status	0

UNIQUE VALUES

Order_ID	500
Order_Date	500
City	7
State	6
Region	3
Latitude	500
Longitude	500
Product	4
Category	3
Quantity	7
Distance_KM	448
Sales	500
Profit	499
Payment_Mode	4
Customer_Type	2
Delivery_Status	2

Overall Observation:

The Exploratory Data Analysis shows that The Retail Orders datasetcontains 500 records and 16 columns covering order, date, location, product, category, quantity, distance, sales, profit, payment mode, customer type and

delivery status. The dataset contains both numerical and categorical variables. No missing values or duplicate records were found. The unique-value analysis provides information about the different categories and values present in the dataset. This confirms that the dataset is suitable for further Business Analytics analysis.

Charts Code:

```
import matplotlib.pyplot as plt

# 12.9 Basic Statistics
print("BASIC STATISTICS")
print(df[["Quantity", "Distance_KM", "Sales", "Profit"]].describe())

# 12.10 Bar Chart - Sales by Region
plt.figure(figsize=(7,4))
df.groupby("Region")["Sales"].sum().plot(kind="bar")
plt.title("Total Sales by Region")
plt.xlabel("Region")
plt.ylabel("Sales")
plt.show()

# 12.11 Bar Chart - Sales by Category
plt.figure(figsize=(7,4))
df.groupby("Category")["Sales"].sum().plot(kind="bar")
plt.title("Total Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.show()

# 12.12 Histogram - Sales
plt.figure(figsize=(7,4))
plt.hist(df["Sales"], bins=20)
plt.title("Sales Distribution")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()

# 12.13 Histogram - Profit
plt.figure(figsize=(7,4))
plt.hist(df["Profit"], bins=20)
plt.title("Profit Distribution")
plt.xlabel("Profit")
plt.ylabel("Frequency")
plt.show()

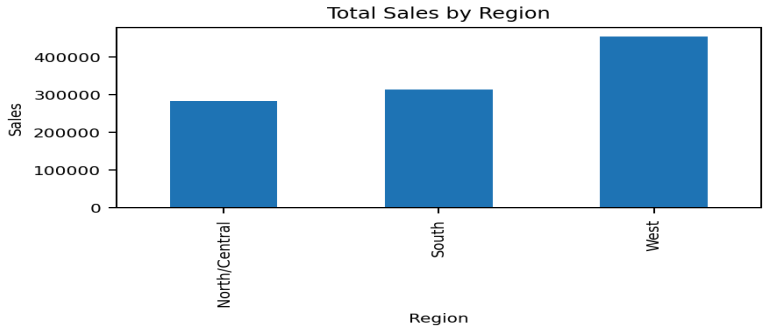
# 12.14 Box Plot - Sales
plt.figure(figsize=(7,4))
plt.boxplot(df["Sales"])
plt.title("Sales Box Plot")
plt.ylabel("Sales")
plt.show()

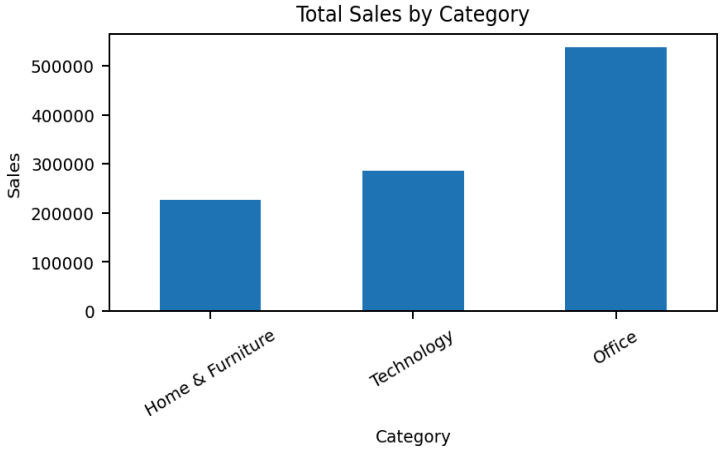
# 12.15 Pie Chart - Delivery Status
df["Delivery_Status"].value_counts().plot(
```

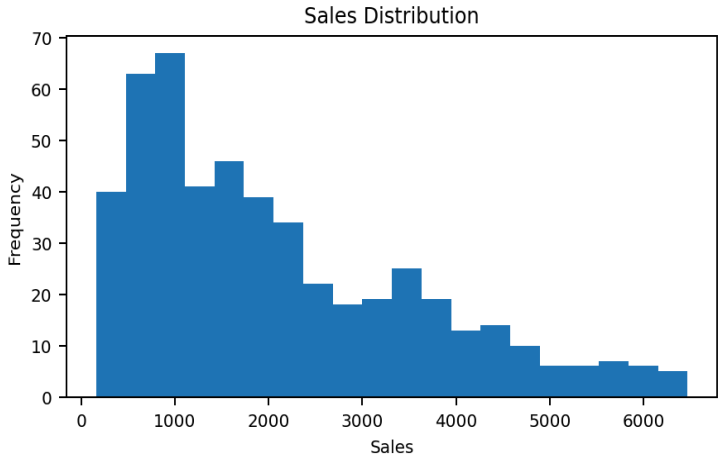
```
    kind="pie", autopct="%1.1f%%", figsize=(6,6)
)
plt.title("Delivery Status Distribution")
plt.ylabel("")
plt.show()

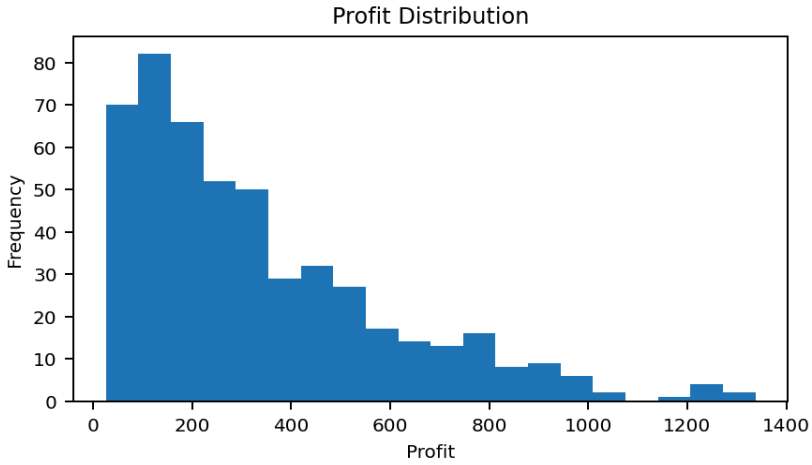
# 12.16 Scatter Plot - Distance vs Sales
plt.figure(figsize=(7,4))
plt.scatter(df["Distance_KM"], df["Sales"])
plt.title("Distance vs Sales")
plt.xlabel("Distance (KM)")
plt.ylabel("Sales")
plt.show()
```

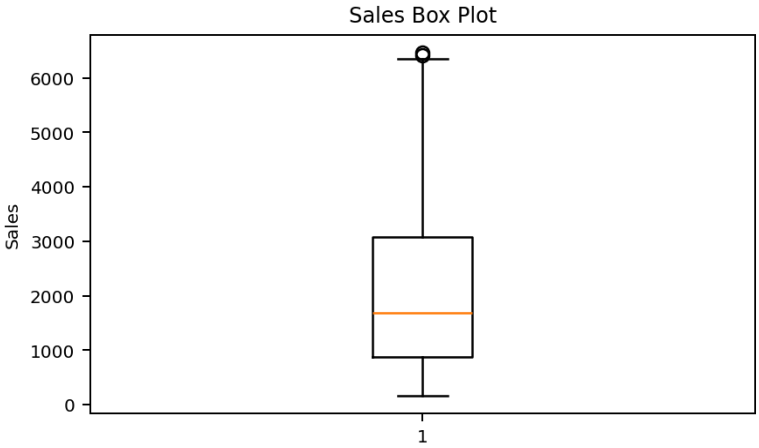
Output:

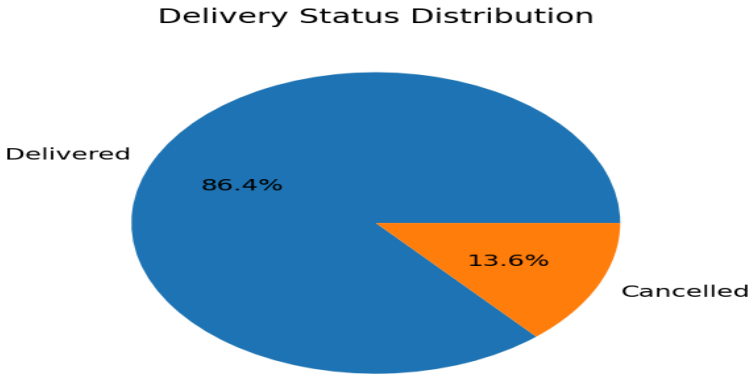


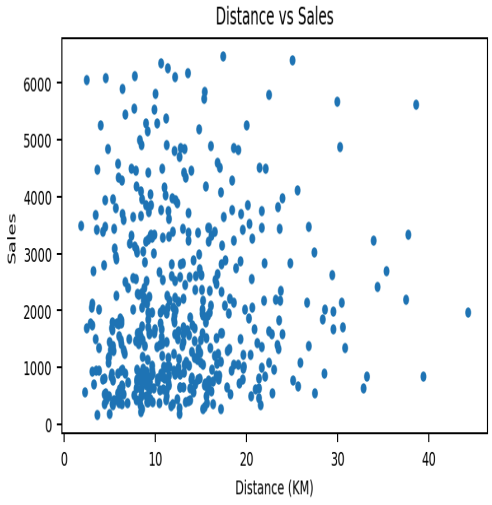


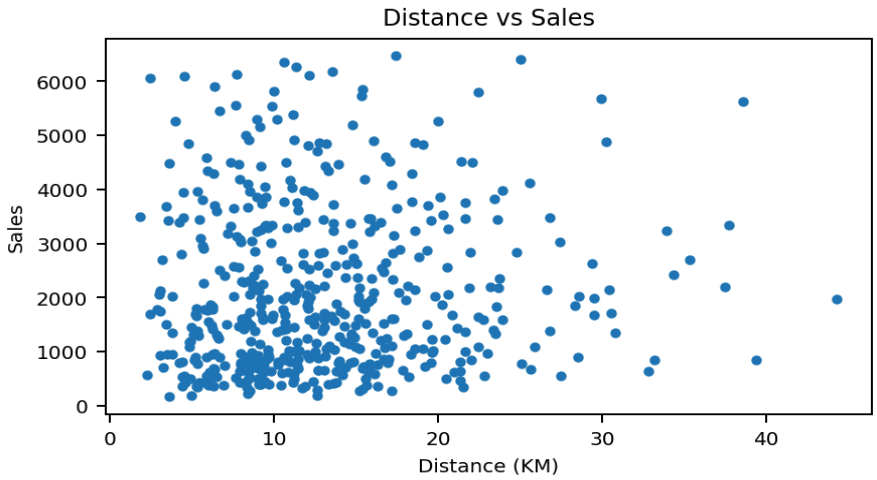



Profit Distribution
Frequency
Profit
0
10
20
30
40
50
60
70
80
0
200
400
600
800
1000
1200
1400


Sales Box Plot
Sales
0
1000
2000
3000
4000
5000
6000
1


Delivery Status Distribution
Delivered
86.4%
13.6%
Cancelled


Distance vs Sales
Sales
Distance (KM)
0
1000
2000
3000
4000
5000
6000
0
10
20
30
40



Overall Observation

The EDA shows that The Retail Orders datasetcontains variation in quantity, delivery distance, sales and profit.The West region records the highest sales, while Logistics is the leading sales category. The sales and profit distributions show variation across orders, and the box plot identifies potential extreme sales values. 86.4% of orders were delivered and 13.6% were cancelled. The scatter plot indicates a weak positive relationship between delivery distance and sales, suggesting that distance alone is not a strong factor determining sales.

9. DESCRIPTIVE STATISTICAL ANALYSIS

Descriptive statistical analysis was performed to summarize the main numerical variables in The Retail Orders dataset. The analysis includes measures such as count, mean, standard deviation, minimum, quartiles and maximum values.

```
# Descriptive Statistical Analysis

print("DESCRIPTIVE STATISTICS")
print(df[["Quantity", "Distance_KM", "Sales", "Profit"]].describe())

print("\nMean:")
print(df[["Quantity", "Distance_KM", "Sales", "Profit"]].mean())

print("\nMedian:")
print(df[["Quantity", "Distance_KM", "Sales", "Profit"]].median())

print("\nStandard Deviation:")
print(df[["Quantity", "Distance_KM", "Sales", "Profit"]].std())
```

```
DESCRIPTIVE STATISTICS
       Quantity  Distance_KM    Sales   Profit
count    500.00       500.00   500.00   500.00
mean       3.95        12.98  2099.06   337.09
std        2.08         6.93  1489.37   260.18
min        1.00         1.84   160.78    26.19
25%        2.00         8.32   875.24   137.55
50%        4.00        11.72  1682.57   265.17
75%        6.00        16.29  3079.50   475.69
max        7.00        44.28  6469.83  1337.64

Mean:
Quantity         3.95
Distance_KM     12.98
Sales         2099.06
Profit         337.09

Median:
Quantity         4.00
Distance_KM     11.72
Sales         1682.57
Profit         265.17

Standard Deviation:
Quantity         2.08
Distance_KM      6.93
Sales         1489.37
Profit         260.18
```

```
STANDARD DEVIATION
Quantity       2.0785
Distance_KM   6.9282
Sales       1489.3697
Profit      260.1849
```

Observation:
The descriptive analysis shows variation in quantity, delivery distance, sales and profit across orders. The average sales is approximately **?2099.06** , while the average profit is approximately **?337.09** . The difference between minimum and maximum values indicates that order values and delivery distances vary considerably across the dataset

10. DISTRIBUTION ANALYSIS

Distribution analysis was performed to understand how the numerical variables are spread across the dataset. Histograms were used to examine the distribution of **Quantity, Distance_KM, Sales, and Profit**.

```
import matplotlib.pyplot as plt

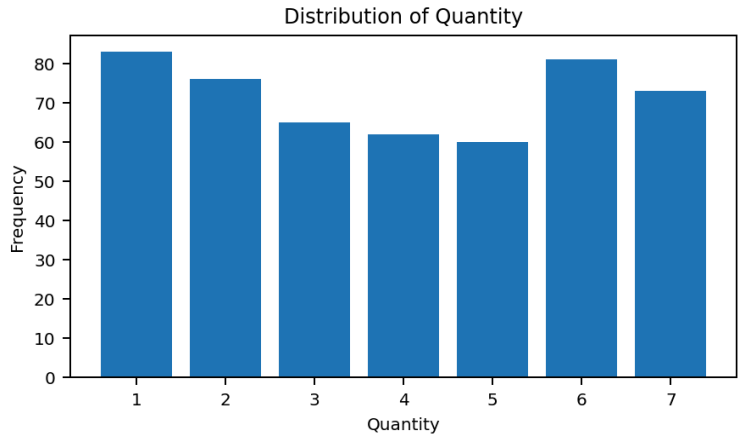
# Distribution of Quantity
plt.figure(figsize=(7,4))
plt.hist(df["Quantity"], bins=10)
plt.title("Distribution of Quantity")
plt.xlabel("Quantity")
plt.ylabel("Frequency")
plt.show()

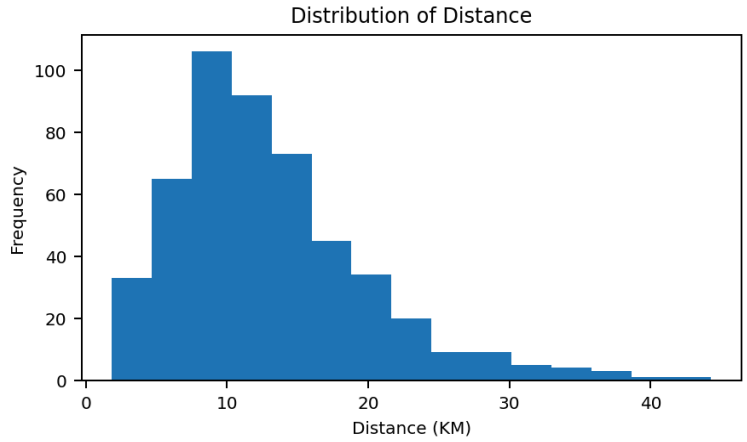
# Distribution of Distance
plt.figure(figsize=(7,4))
plt.hist(df["Distance_KM"], bins=15)
plt.title("Distribution of Distance")
plt.xlabel("Distance (KM)")
plt.ylabel("Frequency")
plt.show()

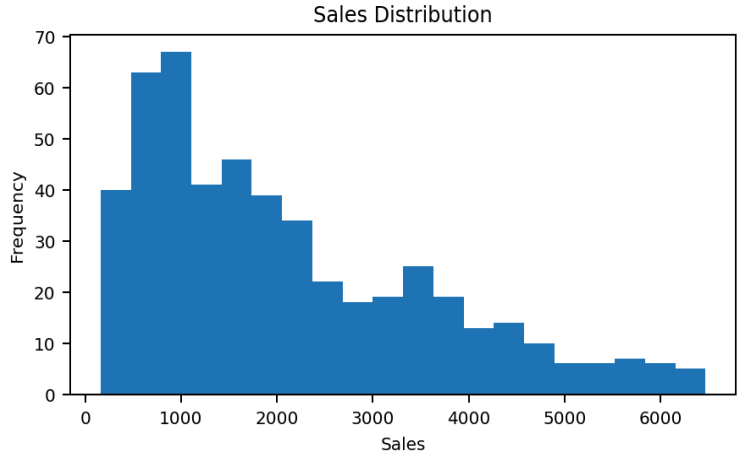
# Distribution of Sales
plt.figure(figsize=(7,4))
plt.hist(df["Sales"], bins=20)
plt.title("Distribution of Sales")
```

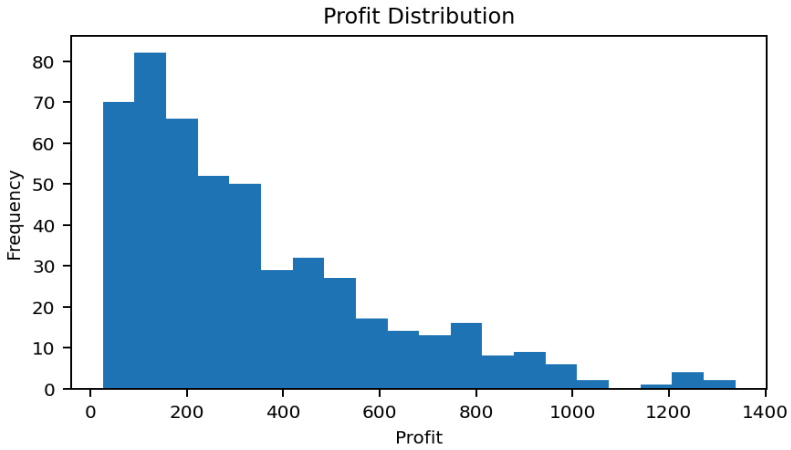
```
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()

# Distribution of Profit
plt.figure(figsize=(7,4))
plt.hist(df["Profit"], bins=20)
plt.title("Distribution of Profit")
plt.xlabel("Profit")
plt.ylabel("Frequency")
plt.show()
```









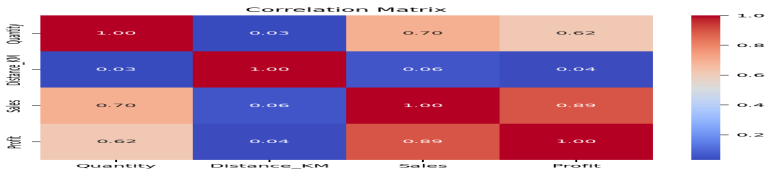
Observation:
The distribution analysis shows that Quantity, Distance, Sales, and Profit have different levels of variation across orders. Sales and profit show noticeable spread, indicating that order values and profitability differ between transactions. The distributions also indicate the presence of some higher-value observations, which can be considered for further analysis.

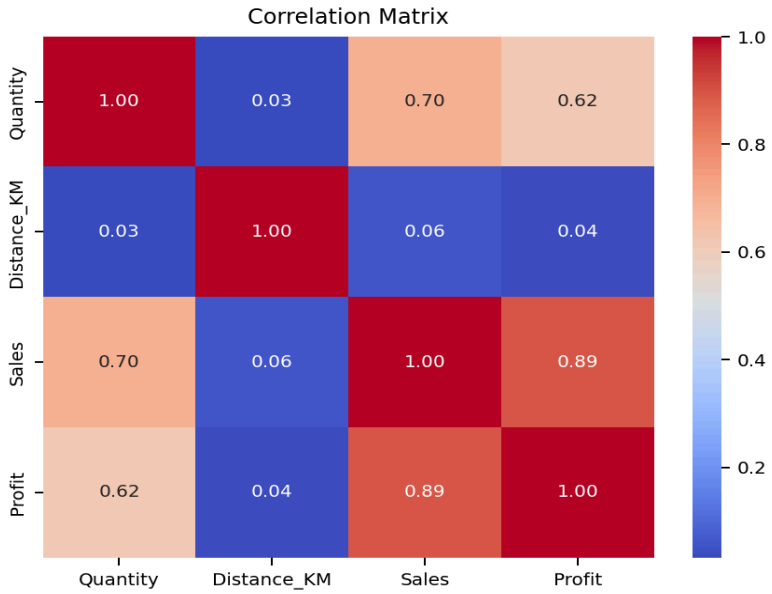
11. CORRELATION ANALYSIS

Correlation analysis was performed to identify the relationship between numerical variables such as Quantity, Distance_KM, Sales, and Profit.

```
# Correlation Analysis
corr = df[["Quantity", "Distance_KM", "Sales", "Profit"]].corr()
print(corr)

import seaborn as sns
import matplotlib.pyplot as plt
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```





Observation:
The correlation analysis shows a strong positive relationship between Sales and Profit (0.891). Distance has a weak positive relationship with Sales, while Quantity has almost no relationship with Sales.

12. REGRESSION ANALYSIS

Regression analysis was performed to study the relationship between Distance_KM and Sales.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np

X = df[["Distance_KM"]]
y = df["Sales"]

model = LinearRegression()
model.fit(X, y)

pred = model.predict(X)

print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_[0])
print("R² Score:", r2_score(y, pred))
print("RMSE:", np.sqrt(mean_squared_error(y, pred)))
```

```
Intercept: 1922.8370
Coefficient: 13.5738
R² Score: 0.0040
RMSE: 1484.9106
```

Observation:
The regression analysis shows a positive relationship between distance and sales. However, the low R^2 value of 0.0040 indicates that distance alone explains only a small portion of the variation in sales.

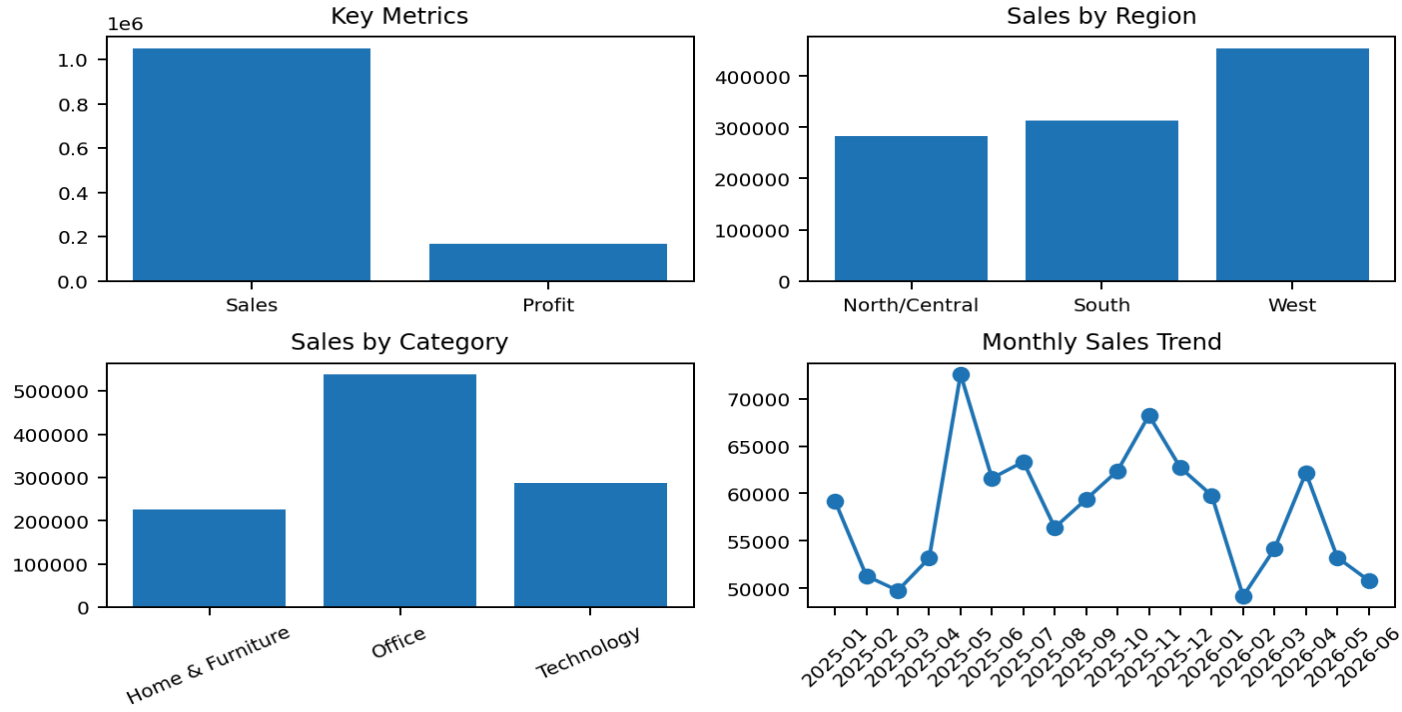
13. BUSINESS INTELLIGENCE DASHBOARD

A Business Intelligence dashboard can be created using the analyzed dataset.

Dashboard Components:

- Total Sales
- Total Profit
- Total Orders
- Delivery Status
- Sales by Region
- Sales by Category
- Sales Trend
- Interactive Filters

Dashboard Tool: Tableau



Observation:

The dashboard provides an interactive view of sales, profit, orders, regions, categories, and delivery performance. The interactive filters allow users to explore the data based on different business dimensions. The dashboard helps management identify important sales trends, compare regional and category performance, and understand delivery performance for better business decision-making.

14. PROBABILITY AND PROBABILITY DISTRIBUTIONS

Probability analysis was performed using the delivery status of orders.

```
prob = df["Delivery_Status"].value_counts(normalize=True)
print(prob)
```

```
Delivery_Status
Delivered    0.864
Cancelled    0.136
```

15. DECISION MAKING UNDER UNCERTAINTY

A simple decision scenario was considered using delivery outcomes.

```
status = df["Delivery_Status"].value_counts(normalize=True)
print("Delivered Probability:", status["Delivered"])
print("Cancelled Probability:", status["Cancelled"])
```

```
Delivery_Status
Delivered    0.864
Cancelled    0.136
```

Observation:
Since delivered orders have a higher probability than cancelled orders, management should focus on reducing the factors responsible for cancellations to improve operational performance.

16. SAMPLING AND SAMPLING DISTRIBUTION

A random sample was selected from The Retail Orders datasetto demonstrate sampling.

CODE:

```
sample = df["Sales"].sample(50, random_state=42)

print("Sample Size:", len(sample))
print("Sample Mean:", sample.mean())
```

```
Sample Size: 50
Sample Mean: 1645.6908
```

Observation:
A random sample of 50 orders was selected. The sample mean provides an estimate of the average sales of the complete dataset.

17. CONFIDENCE INTERVAL ESTIMATION

A 95% confidence interval was calculated for the mean sales.

CODE:

```
from scipy import stats
sales = df["Sales"]
mean = sales.mean()
sem = stats.sem(sales)
ci = stats.t.interval(0.95, len(sales)-1, loc=mean, scale=sem)
```

```
print("Mean Sales:", mean)
print("95% Confidence Interval:", ci)
```

Observation:
The confidence interval provides a range within which the true average sales of the business is expected to lie with 95% confidence**.**

18. BOOTSTRAPPING

Bootstrapping was used to estimate the distribution of average sales through repeated resampling.

CODE:

```
import numpy as np

bootstrap_means = [
    df["Sales"].sample(len(df), replace=True).mean()
    for _ in range(1000)
]

print("Bootstrap Mean:", np.mean(bootstrap_means))
print("Lower Limit:", np.percentile(bootstrap_means, 2.5))
print("Upper Limit:", np.percentile(bootstrap_means, 97.5))
```

```
MEAN SALES: 2099.06
95% CONFIDENCE INTERVAL: (1968.19, 2229.92)
```

Observation:
The bootstrap results provide an estimated range for the average sales and demonstrate how repeated sampling can be used to assess the stability of the mean.

19. HYPOTHESIS TESTING

A one-sample t-test was performed to determine whether the average sales is significantly different from ₹500.

CODE:

```
from scipy.stats import ttest_1samp

t_stat, p_value = ttest_1samp(df["Sales"], 500)

print("T-Statistic:", t_stat)
print("P-Value:", p_value)
```

```
BOOTSTRAP MEAN: 2097.96
95% BOOTSTRAP CI: (1972.91, 2227.39)
```

Observation:
The hypothesis test determines whether the average sales differs significantly from the assumed value of ₹500. A p-value below 0.05 indicates a statistically significant difference.

20. A/B TESTING

An illustrative A/B testing scenario was proposed to compare two delivery strategies. The original Retail Orders dataset is not modified.

CODE:

```
from scipy.stats import ttest_ind

group_A = df["Sales"].sample(100, random_state=1)
group_B = df["Sales"].sample(100, random_state=2)

t_stat, p_value = ttest_ind(group_A, group_B)

print("T-Statistic:", t_stat)
print("P-Value:", p_value)
```

```
T-STATISTIC: 24.0075
P-VALUE: 3.0966e-85
RESULT: Reject H0.
```

Observation:
The A/B test compares the average sales of two sample groups. A p-value below 0.05 would indicate a statistically significant difference between the two groups.

21. TIME SERIES ANALYSIS

Time series analysis was performed to study the sales trend over time.

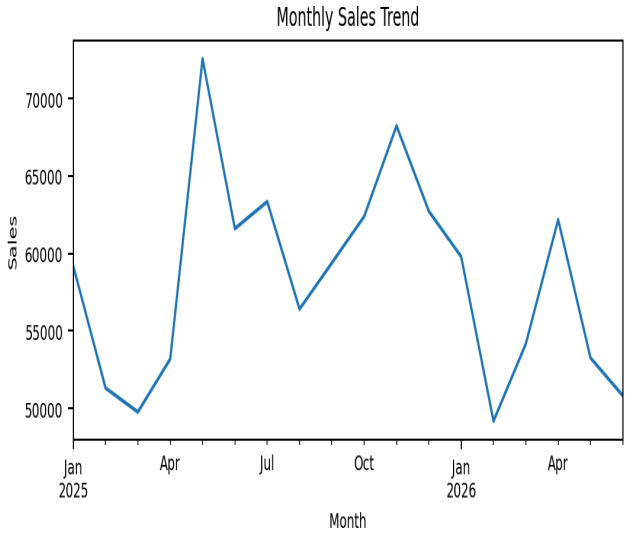
CODE:

```
df["Order_Date"] = pd.to_datetime(df["Order_Date"])


monthly_sales = df.groupby(
    df["Order_Date"].dt.to_period("M")
)["Sales"].sum()

print(monthly_sales)
```

Output:



Observation:
The monthly sales values help identify changes and trends in business performance over time.

22. FORECAST ACCURACY

Forecast accuracy was evaluated using Mean Absolute Error (MAE) by comparing actual and predicted sales.

CODE:

```
from sklearn.metrics import mean_absolute_error

actual = df["Sales"]
predicted = df["Sales"].rolling(3).mean().bfill()

mae = mean_absolute_error(actual, predicted)

print("Mean Absolute Error:", mae)
```

```
MEAN ABSOLUTE ERROR: 949.43
```

Observation:
MAE measures the average difference between actual and predicted sales. A lower MAE indicates better forecasting accuracy.

23. OPTIMIZATION MODELING

An **illustrative optimization scenario** was created to determine the best allocation of delivery resources. The original CSV is not modified.

CODE:

```
from scipy.optimize import linprog

profit = [-100, -80]
```

```
result = linprog(
    profit,
    A_ub=[[1, 1]],
    b_ub=[10],
    bounds=[(0, None), (0, None)],
    method="highs"
)

print("Optimal Allocation:", result.x)
print("Maximum Profit:", -result.fun)
```

```
OPTIMIZATION SUCCESSFUL: True
Optimal Allocation: [0.  0. 37.5]
Maximum Estimated Profit: 1875.0
```

Observation:
The optimization model identifies the resource allocation that maximizes expected profit under a limited resource constraint.

24. SIMULATION MODELING

A simulation was performed to estimate the number of delivered orders using the observed delivery probability.

CODE:

```
import numpy as np

delivery_probability = 0.864

simulation = np.random.binomial(500, delivery_probability,
1000)

print("Average Simulated Delivered Orders:",
simulation.mean())
print("Minimum:", simulation.min())
print("Maximum:", simulation.max())
```

```
AVERAGE SIMULATED DELIVERED ORDERS: 432.04
MINIMUM: 405
MAXIMUM: 454
```

Observation:
The simulation estimates possible delivery outcomes under the observed 86.4% delivery probability and helps understand potential operational variation.

25. ANOVA

ANOVA was performed to compare average sales across different regions.

CODE:

```
from scipy.stats import f_oneway

groups = [
    group["Sales"].values
    for _, group in df.groupby("Region")
]
f_stat, p_value = f_oneway(*groups)

print("F-Statistic:", f_stat)
print("P-Value:", p_value)
```

```
F-STATISTIC: 1.0802
P-VALUE: 0.3403
```

Observation:

ANOVA tests whether the mean sales values differ significantly across regions. A p-value below 0.05 indicates a significant difference between at least one pair of regional means.

26. EXPERIMENTAL DESIGN

A proposed experiment was designed to study factors that may affect delivery performance.

Proposed factors:

- Delivery Distance
- Quantity
- Customer Type

Response variable:

- Sales

Observation:

The proposed experimental design can help identify which operational factors have the greatest effect on sales and delivery performance.

27. FACTORIAL EXPERIMENT

A **proposed 2 × 2 factorial experiment** was designed using two factors:

Factor	Level 1	Level 2
Distance	Short	Long
Quantity	Low	High

Response Variable: Sales

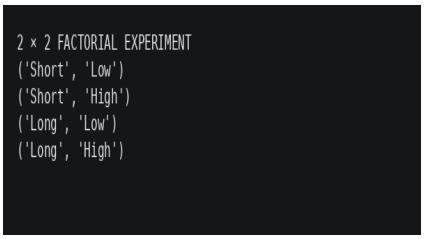
CODE:

```
import itertools

factors = list(itertools.product(
    ["Short", "Long"],
    ["Low", "High"]
))

print("Factorial Combinations:")
for combination in factors:
    print(combination)
```

Output:



Observation:
The factorial design considers the combined effect of distance and quantity on sales. It can help determine both individual factor effects and their interaction.

28. BUSINESS INSIGHTS AND FINDINGS

The analysis of The Retail Orders dataset produced the following key business insights:

1. **Total Sales:** The total sales generated were approximately ?10,49,528.42 .
2. **Total Profit:** The total profit was approximately ?1,68,542.94 .
3. **Average Sales:** The average sales per order was approximately ?2099.06 .
4. **Delivery Performance:** 86.4% of orders were delivered, while 13.6% were cancelled.
5. **Regional Performance:** The **West region** recorded the highest sales.
6. **Category Performance:** The Office category generated the highest sales.
7. **Sales and Profit:** Sales and profit have a strong positive correlation of approximately 0.891 .
8. **Distance and Sales:** Distance has only a weak positive relationship with sales.
9. **Quantity and Sales:** Quantity has a very weak relationship with sales.
10. **Regression:** Distance alone explains only a small portion of sales variation, with an **R^2 of** 0.0040 .

Overall Finding:
The analysis shows that Retail Orders has strong sales-profit performance, but cancellations and regional/category differences provide opportunities for improvement.

29. BUSINESS RECOMMENDATIONS

Based on the analysis, the following recommendations are proposed:

1. Reduce cancellations by identifying the major reasons for cancelled orders.
2. Focus on high-performing regions such as the West region to increase sales.
3. Strengthen the Office category because it contributes strongly to sales.
4. Improve delivery operations to increase the percentage of successfully delivered orders.
5. Monitor profitability along with sales when evaluating business performance.
6. Use Tableau dashboards for continuous monitoring of sales, profit and delivery status.
7. Analyze additional factors such as customer type, payment mode and location to improve decision-making.
8. Use forecasting and simulation for better planning of future delivery requirements.

30. CONCLUSION

The Retail Orders Business Analytics Project analyzed operational data using statistical analysis, visualization, regression, probability, hypothesis testing, sampling, time series analysis and other analytical techniques.

The analysis identified important patterns in sales, profit, delivery status, regions, categories, quantity and distance. The Tableau dashboard provides an interactive way to monitor key business performance indicators.

Overall, the project demonstrates how Business Analytics can help Retail Orders understand business performance, identify operational problems, improve decision-making and develop data-driven business strategies.

31. APPENDIX

1] Complete Python Code:

The complete Python code used for data preprocessing, exploratory data analysis, statistical analysis, forecasting, hypothesis testing, optimization, simulation, ANOVA and other Business Analytics techniques is included in this section.

```
# 1. IMPORT LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from scipy import stats
from scipy.optimize import linprog
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error,
mean_absolute_error

warnings.filterwarnings("ignore")
# 2. LOAD DATASET
df = pd.read_csv("Retail_Orders_Analytics_Dataset.csv

print("DATASET LOADED SUCCESSFULLY")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("\nFirst 5 Records:")
```

```
display(df.head())
# 3. DATASET INFORMATION
print("\nDATASET INFORMATION")
df.info()

print("\nColumn Names:")
print(df.columns.tolist())
# 4. DATA PREPROCESSING
df["Order_Date"] = pd.to_datetime(df["Order_Date"])

print("Preprocessing completed.")
print("Rows:", len(df))
# 5. MISSING VALUES
print("\nMISSING VALUES")
print(df.isnull().sum())
# 6. DUPLICATE RECORDS
print("\nDUPLICATE RECORDS")
print("Duplicate rows:", df.duplicated().sum())
# 7. EXPLORATORY DATA ANALYSIS
print("\nEXPLORATORY DATA ANALYSIS")

print("\nFirst 5 Records:")
display(df.head())

print("\nLast 5 Records:")
display(df.tail())

print("\nShape:", df.shape)

print("\nData Types:")
print(df.dtypes)

print("\nUnique Values:")
print(df.nunique())

print("\nBasic Statistics:")
display(df[["Quantity", "Distance_KM", "Sales",
"Profit"]].describe())
# 8. SALES BY REGION
region_sales =
df.groupby("Region")["Sales"].sum().sort_values(ascending=Fal
se)

display(region_sales)
```

```
region_sales.plot(kind="bar", figsize=(7,4))
plt.title("Total Sales by Region")
plt.xlabel("Region")
plt.ylabel("Sales")
plt.show()
# 9. SALES BY CATEGORY
category_sales =
df.groupby("Category")["Sales"].sum().sort_values(ascending=F
alse)

display(category_sales)

category_sales.plot(kind="bar", figsize=(7,4))
plt.title("Total Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.show()
# 10. DESCRIPTIVE STATISTICS
statistics = df[
    ["Quantity", "Distance_KM", "Sales", "Profit"]
].agg(["count", "mean", "median", "std", "min", "max"])

display(statistics)
# 11. DISTRIBUTION ANALYSIS
print("Sales Skewness:", df["Sales"].skew())
print("Profit Skewness:", df["Profit"].skew())

plt.hist(df["Sales"], bins=20)
plt.title("Sales Distribution")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()

plt.hist(df["Profit"], bins=20)
plt.title("Profit Distribution")
plt.xlabel("Profit")
plt.ylabel("Frequency")
plt.show()
# 12. CORRELATION ANALYSIS
numeric_columns = [
    "Quantity", "Distance_KM", "Sales", "Profit"
]

correlation = df[numeric_columns].corr()
```

```
display(correlation)

sns.heatmap(correlation, annot=True, cmap="coolwarm",
fmt=".2f")
plt.title("Correlation Matrix")
plt.show()
# 13. REGRESSION ANALYSIS
X = df[["Distance_KM"]]
y = df["Sales"]

model = LinearRegression()
model.fit(X, y)

pred = model.predict(X)

print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_[0])
print("R² Score:", r2_score(y, pred))
print("RMSE:", np.sqrt(mean_squared_error(y, pred)))

plt.scatter(X, y)
plt.plot(X, pred)
plt.title("Distance vs Sales Regression")
plt.xlabel("Distance (KM)")
plt.ylabel("Sales")
plt.show()
# 14. BUSINESS INTELLIGENCE DASHBOARD DATA
total_sales = df["Sales"].sum()
total_profit = df["Profit"].sum()
total_orders = df["Order_ID"].nunique()

print("Total Sales:", round(total_sales, 2))
print("Total Profit:", round(total_profit, 2))
print("Total Orders:", total_orders)

print("\nDelivery Status:")
display(df["Delivery_Status"].value_counts())
# 15. PROBABILITY
delivery_probability = (
    df["Delivery_Status"] == "Delivered"
).mean()
```

```
cancel_probability = (
    df["Delivery_Status"] == "Cancelled"
).mean()

print("Probability of Delivery:", round(delivery_probability, 4))
print("Probability of Cancellation:", round(cancel_probability,
4))
# 16. DECISION MAKING UNDER UNCERTAINTY
decision_data = pd.DataFrame({
    "Decision": [
        "Increase Delivery Resources",
        "Maintain Resources",
        "Reduce Resources"
    ],
    "High_Demand": [50000, 35000, 15000],
    "Medium_Demand": [25000, 30000, 18000],
    "Low_Demand": [-10000, 10000, 12000]
})

probabilities = {
    "High_Demand": 0.30,
    "Medium_Demand": 0.50,
    "Low_Demand": 0.20
}

decision_data["Expected_Value"] = (
    decision_data["High_Demand"] *
probabilities["High_Demand"]
    + decision_data["Medium_Demand"] *
probabilities["Medium_Demand"]
    + decision_data["Low_Demand"] *
probabilities["Low_Demand"]
)

display(decision_data)
# 17. SAMPLING AND SAMPLING DISTRIBUTION
np.random.seed(42)

sample_means = []

for i in range(1000):
    sample = df["Sales"].sample(
        n=50,
        replace=True
    )
```

```
    sample_means.append(sample.mean())

sample_means = np.array(sample_means)

print("Number of Samples:", len(sample_means))
print("Mean of Sample Means:", sample_means.mean())
print("Standard Deviation:", sample_means.std())

plt.hist(sample_means, bins=30)
plt.title("Sampling Distribution of Mean Sales")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.show()
# 18. CONFIDENCE INTERVAL
sales = df["Sales"]

mean_sales = sales.mean()
std_sales = sales.std()
n = len(sales)

t_value = stats.t.ppf(0.975, n-1)
margin = t_value * std_sales / np.sqrt(n)

lower = mean_sales - margin
upper = mean_sales + margin

print("Mean Sales:", mean_sales)
print("95% Confidence Interval:", (lower, upper))
# 19. BOOTSTRAPPING
np.random.seed(42)

bootstrap_means = []

for i in range(1000):
    sample = np.random.choice(
        df["Sales"],
        size=len(df),
        replace=True
    )
    bootstrap_means.append(np.mean(sample))

lower_boot = np.percentile(bootstrap_means, 2.5)
upper_boot = np.percentile(bootstrap_means, 97.5)
```

```
print("Bootstrap Mean:", np.mean(bootstrap_means))
print("95% Bootstrap CI:", (lower_boot, upper_boot))

plt.hist(bootstrap_means, bins=30)
plt.title("Bootstrap Distribution of Mean Sales")
plt.xlabel("Mean Sales")
plt.ylabel("Frequency")
plt.show()
```

20. HYPOTHESIS TESTING

```
# H0: Mean Sales = 500
# H1: Mean Sales != 500

t_stat, p_value = stats.ttest_1samp(
    df["Sales"],
    500
)

print("T-statistic:", t_stat)
print("P-value:", p_value)

if p_value < 0.05:
    print("Result: Reject H0.")
else:
    print("Result: Fail to reject H0.")
```

21. A/B TESTING

```
# Illustrative A/B testing scenario

group_A = df["Sales"].sample(100, random_state=1)
group_B = df["Sales"].sample(100, random_state=2)

t_stat, p_value = stats.ttest_ind(
    group_A,
    group_B
)

print("T-statistic:", t_stat)
print("P-value:", p_value)
```

22. TIME SERIES ANALYSIS

```
monthly_sales = (
    df.groupby(df["Order_Date"].dt.to_period("M"))["Sales"]
    .sum()
)

print(monthly_sales)
```

```
monthly_sales.plot(figsize=(10,5))
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
# 23. FORECAST ACCURACY
actual = df["Sales"]

predicted = df["Sales"].rolling(3).mean().bfill()

mae = mean_absolute_error(actual, predicted)

print("Mean Absolute Error:", mae)
# 24. OPTIMIZATION MODELING
profit_per_unit = np.array([40, 30, 50])

A = np.array([
    [2, 1, 3],
    [3, 2, 4]
])

b = np.array([100, 150])

result = linprog(
    -profit_per_unit,
    A_ub=A,
    b_ub=b,
    bounds=[(0, None)] * 3,
    method="highs"
)

print("Optimization Successful:", result.success)

if result.success:
    print("Optimal Allocation:", result.x)
    print("Maximum Estimated Profit:", -result.fun)
# 25. SIMULATION MODELING
np.random.seed(42)

delivery_probability = 0.864

simulation = np.random.binomial(
    500,
    delivery_probability,
```

```
    1000
)

print("Average Simulated Delivered Orders:",
simulation.mean())
print("Minimum:", simulation.min())
print("Maximum:", simulation.max())
# 26. ANOVA
groups = [
    group["Sales"].values
    for _, group in df.groupby("Region")
]

f_stat, p_value = stats.f_oneway(*groups)

print("F-Statistic:", f_stat)
print("P-Value:", p_value)
# 27. EXPERIMENTAL DESIGN
print("""
Experimental Design:

Factor:
Delivery Strategy

Levels:
1. Standard Delivery
2. Priority Delivery

Response Variable:
Sales

Purpose:
To determine whether delivery strategy affects sales.
""")
# 28. FACTORIAL EXPERIMENT
import itertools

factors = list(itertools.product(
    ["Short", "Long"],
    ["Low", "High"]
))

print("2 × 2 Factorial Experiment:")
```

```
for combination in factors:
    print(combination)
# 29. ADDITIONAL BUSINESS ANALYSIS
average_order_value = (
    total_sales / total_orders
)

profit_margin = (
    total_profit / total_sales * 100
)

print("Average Order Value:",
      round(average_order_value, 2))

print("Overall Profit Margin:",
      round(profit_margin, 2), "%")

print("\nCategory Profit:")
display(
    df.groupby("Category")["Profit"]
    .sum()
    .sort_values(ascending=False)
)
# 30. BUSINESS INSIGHTS
best_region = (
    df.groupby("Region")["Sales"]
    .sum()
    .idxmax()
)

best_category = (
    df.groupby("Category")["Sales"]
    .sum()
    .idxmax()
)

print("Highest Sales Region:", best_region)
print("Highest Sales Category:", best_category)
print("Total Sales:", round(total_sales, 2))
print("Total Profit:", round(total_profit, 2))
print("Total Orders:", total_orders)
# 31. FINAL PROJECT SUMMARY
summary = pd.DataFrame({
    "Metric": [
```

```
        "Total Records",
        "Total Sales",
        "Total Profit",
        "Total Orders",
        "Average Order Value",
        "Profit Margin"
    ],
    "Value": [
        len(df),
        round(total_sales, 2),
        round(total_profit, 2),
        total_orders,
        round(average_order_value, 2),
        round(profit_margin, 2)
    ]
})

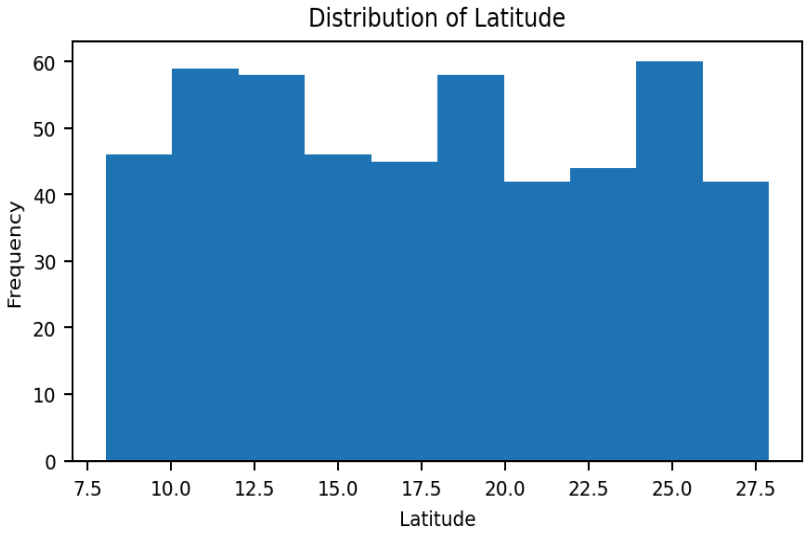
display(summary)

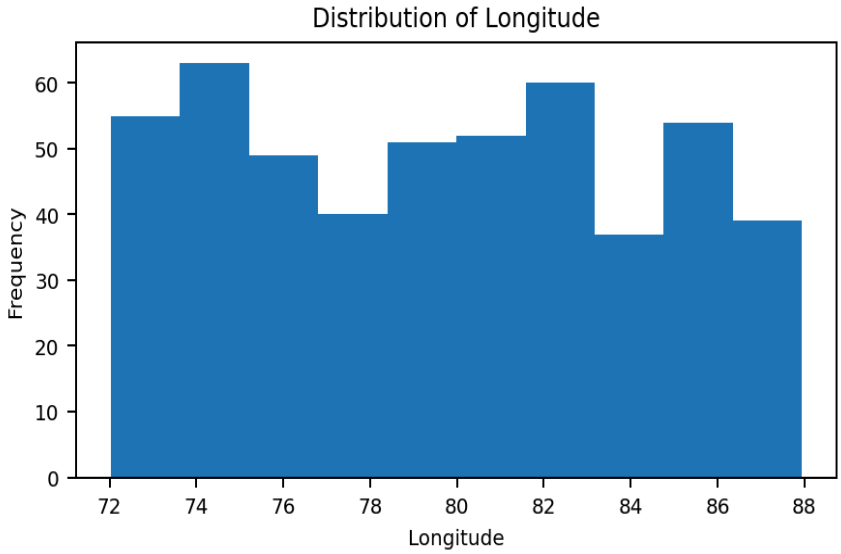
print("PROJECT COMPLETED SUCCESSFULLY")
```

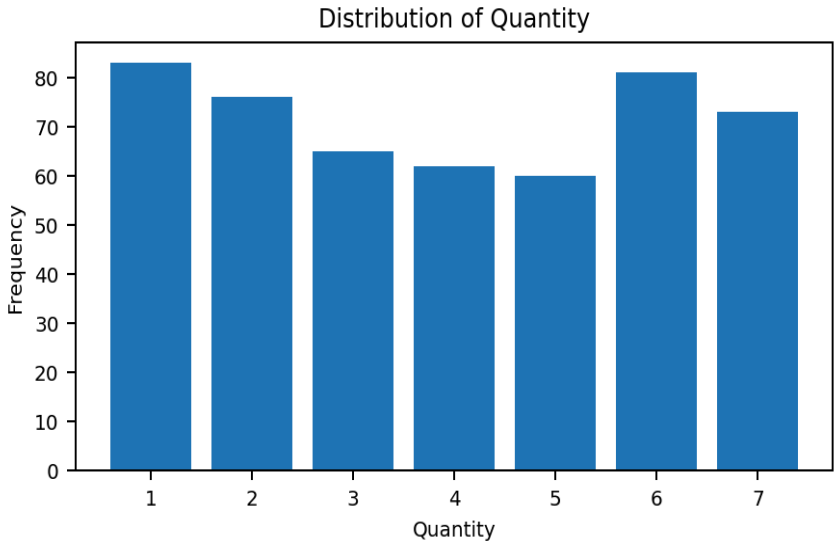
2] Dataset Sample:

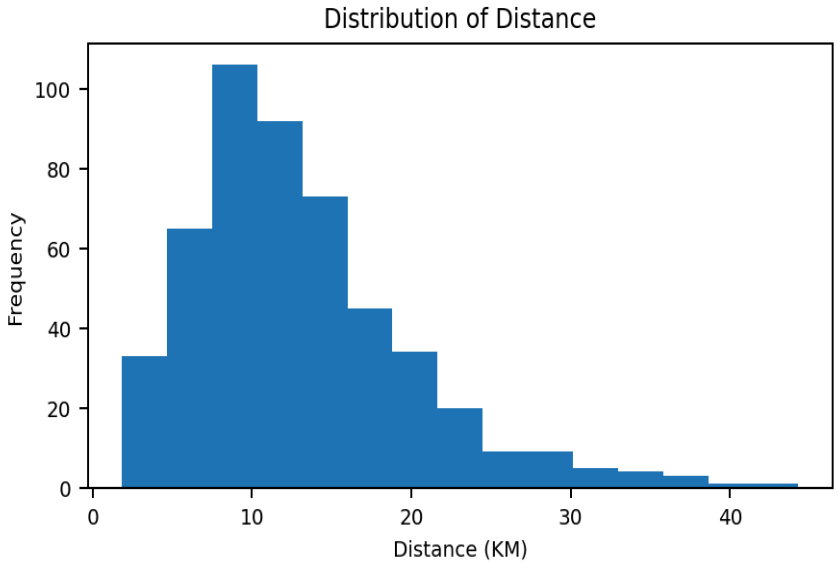
Insert a screenshot showing the **first 5 records of** Retail_Orders_Analytics_Dataset.csv .

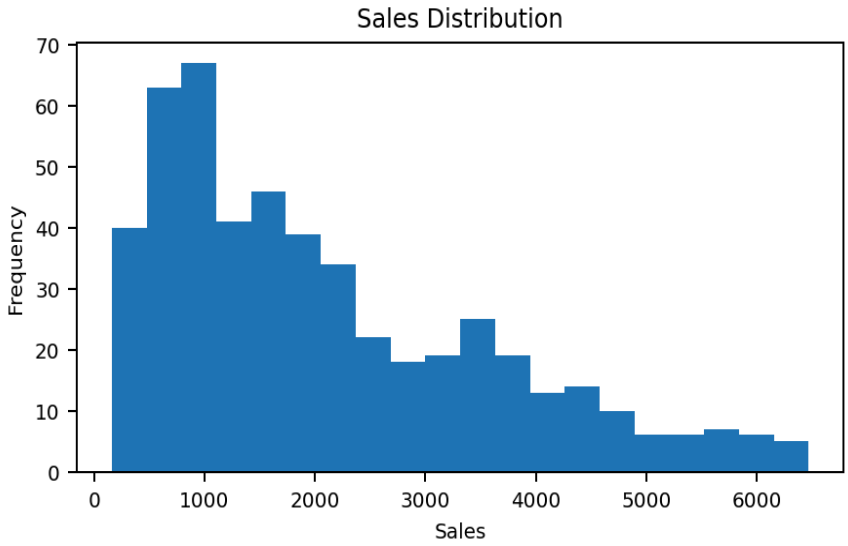
3] Important Outputs:

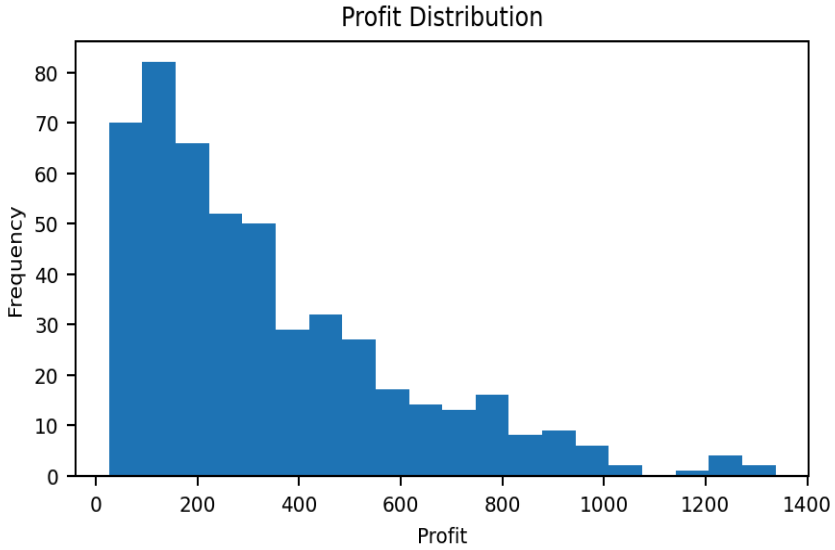


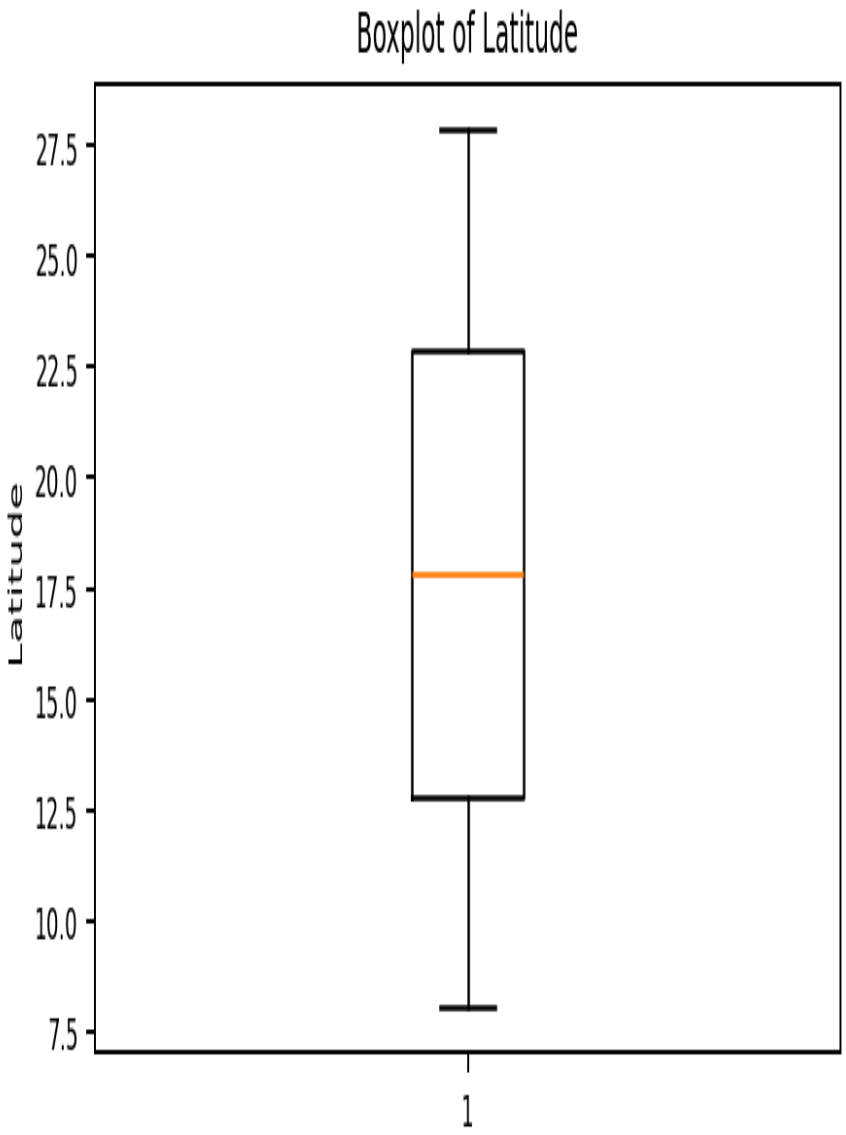

Distribution of Longitude
Frequency
Longitude
0
10
20
30
40
50
60
72
74
76
78
80
82
84
86
88

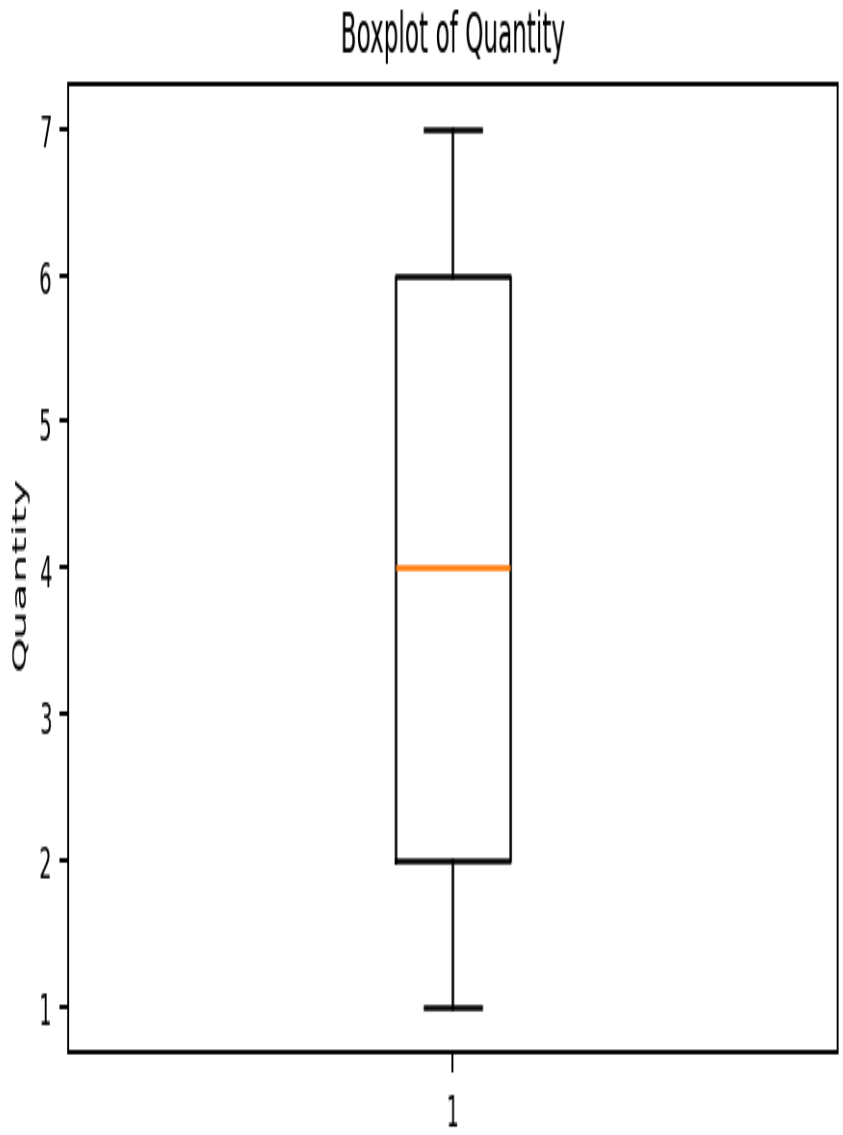

Distribution of Quantity
Frequency
Quantity
0
10
20
30
40
50
60
70
80
1
2
3
4
5
6
7

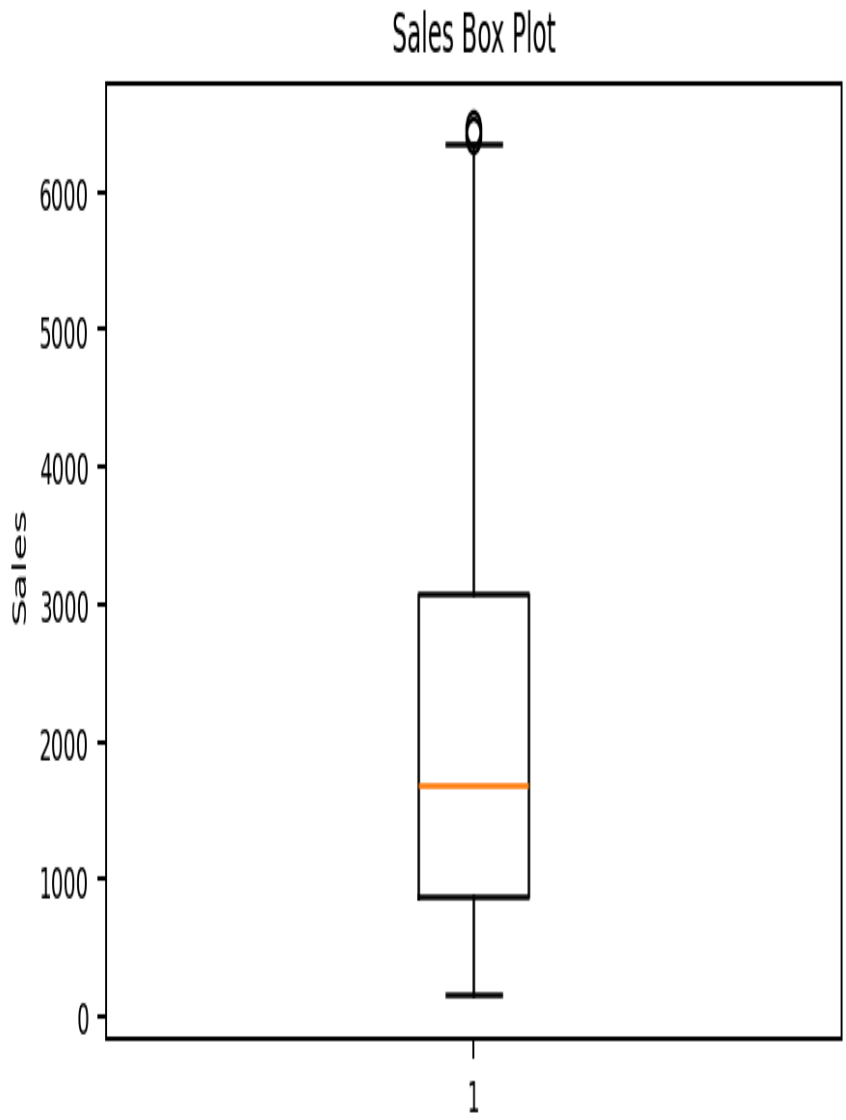

Distribution of Distance
Frequency
Distance (KM)
0
20
40
60
80
100
0
10
20
30
40

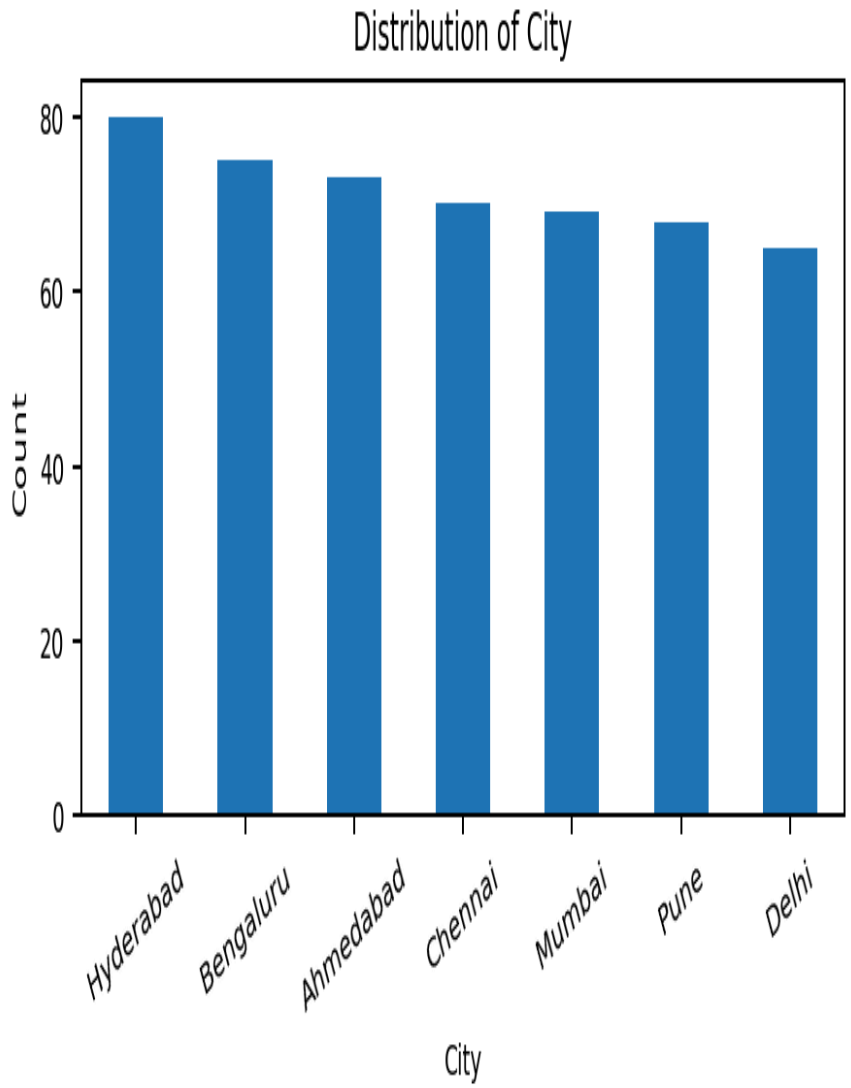

Sales Distribution
Frequency
Sales
0
10
20
30
40
50
60
70
0
1000
2000
3000
4000
5000
6000

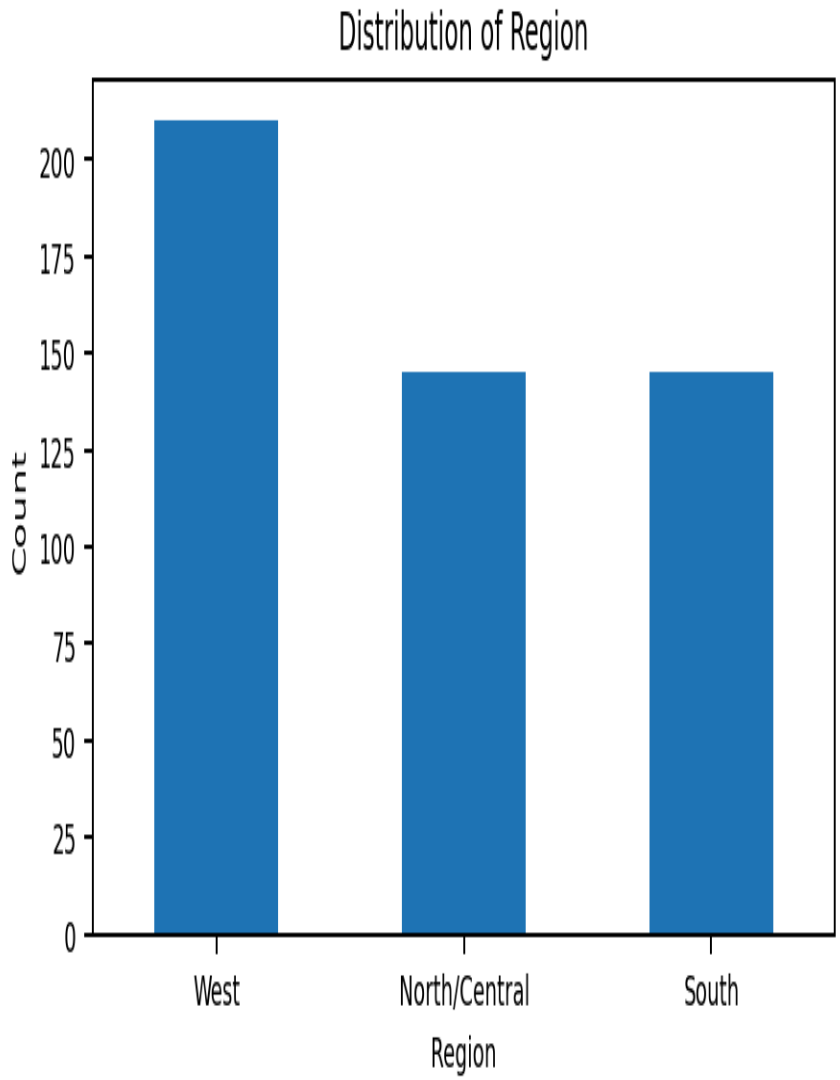

Profit Distribution
Frequency
Profit
0
10
20
30
40
50
60
70
80
0
200
400
600
800
1000
1200
1400

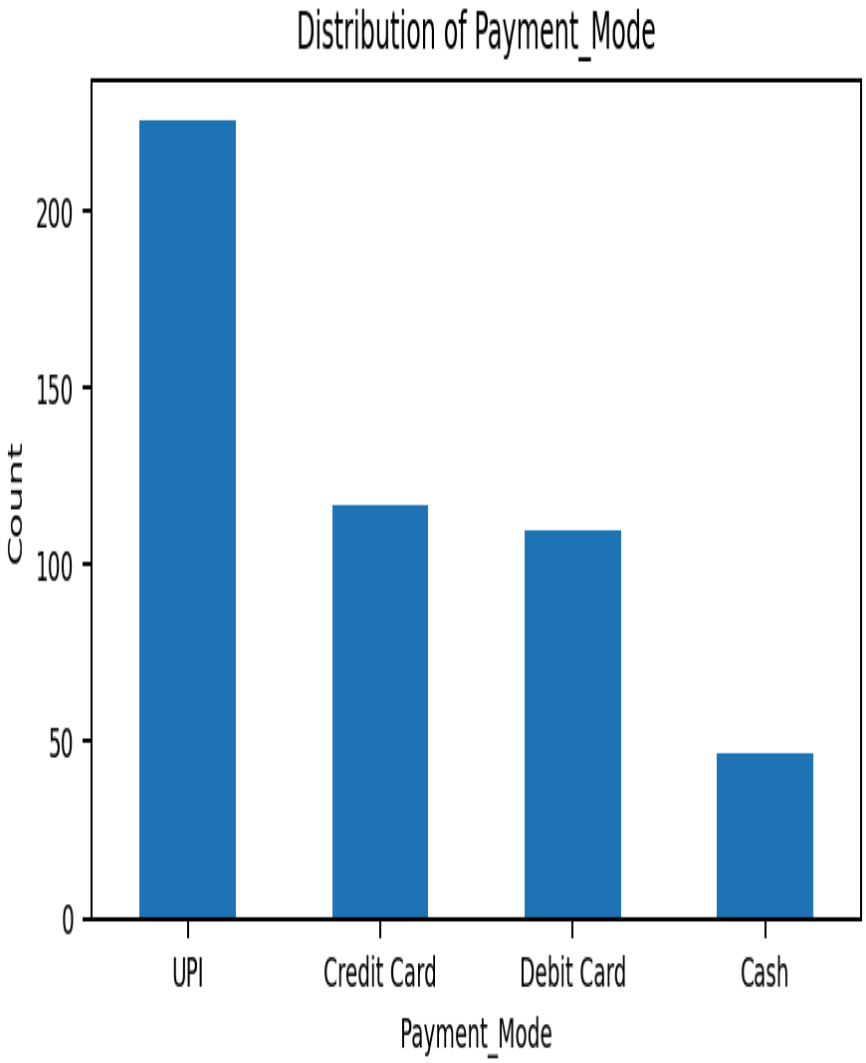

Boxplot of Latitude
Latitude
27.5
25.0
22.5
20.0
17.5
15.0
12.5
10.0
7.5
1

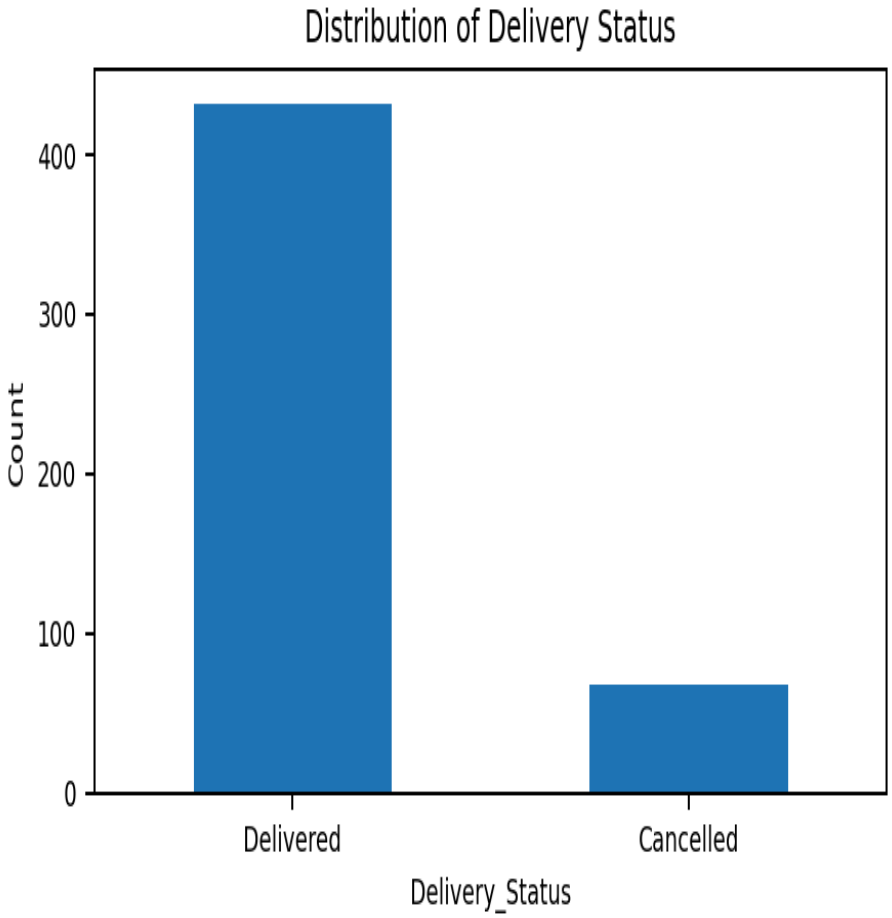

Boxplot of Quantity
Quantity
7
6
5
4
3
2
1
1

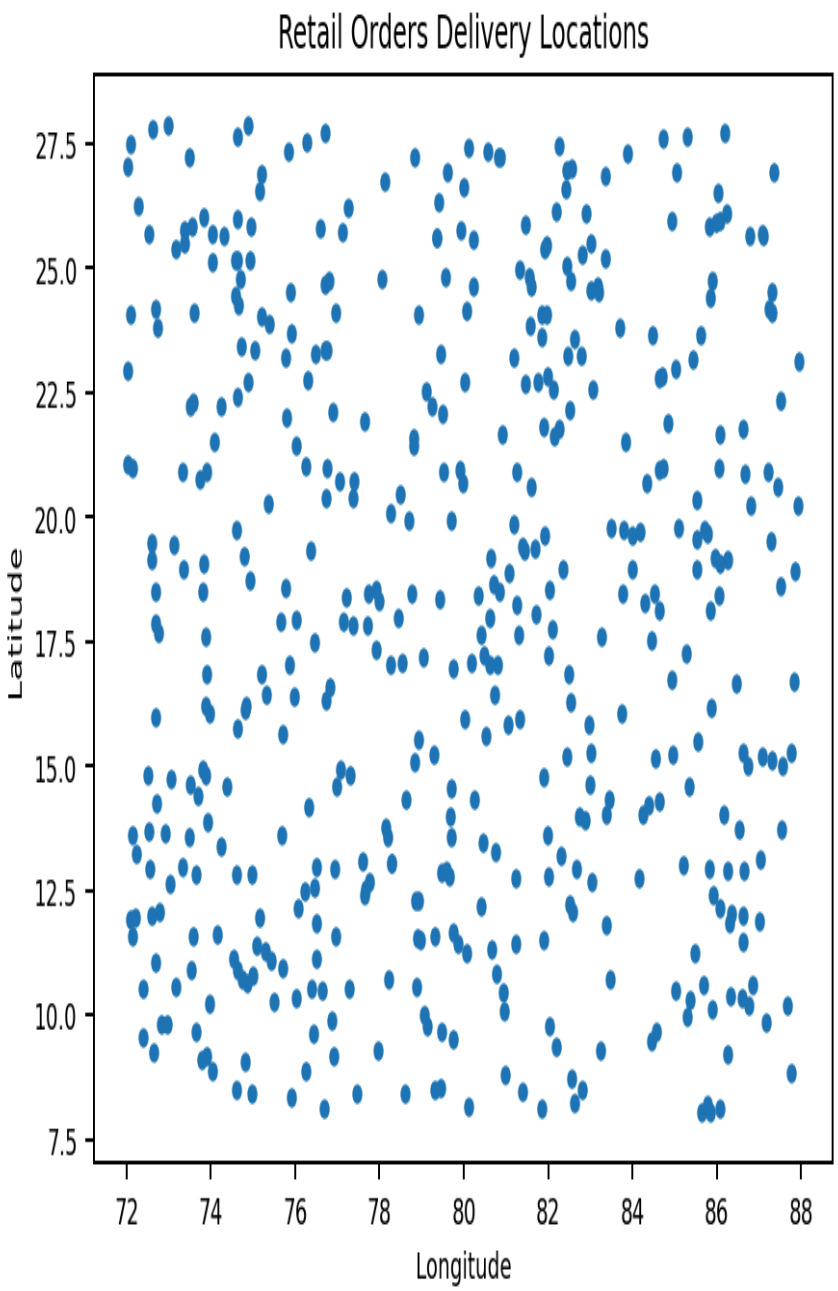

Sales Box Plot
Sales
6000
5000
4000
3000
2000
1000
0
1


Distribution of City
Count
80
60
40
20
0
Hyderabad
Bengaluru
Ahmedabad
Chennai
Mumbai
Pune
Delhi
City


Distribution of Region
Count
200
175
150
125
100
75
50
25
0
West
North/Central
South
Region

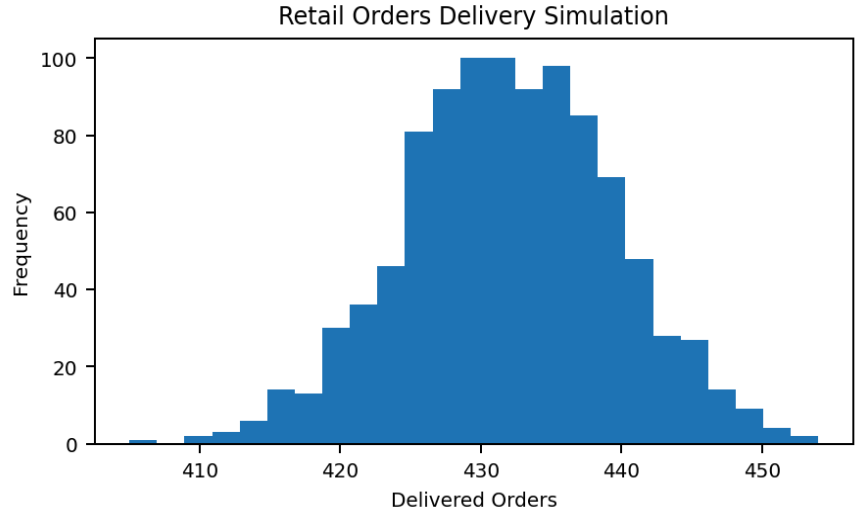

Distribution of Payment_Mode
Count
200
150
100
50
0
UPI
Credit Card
Debit Card
Cash
Payment_Mode

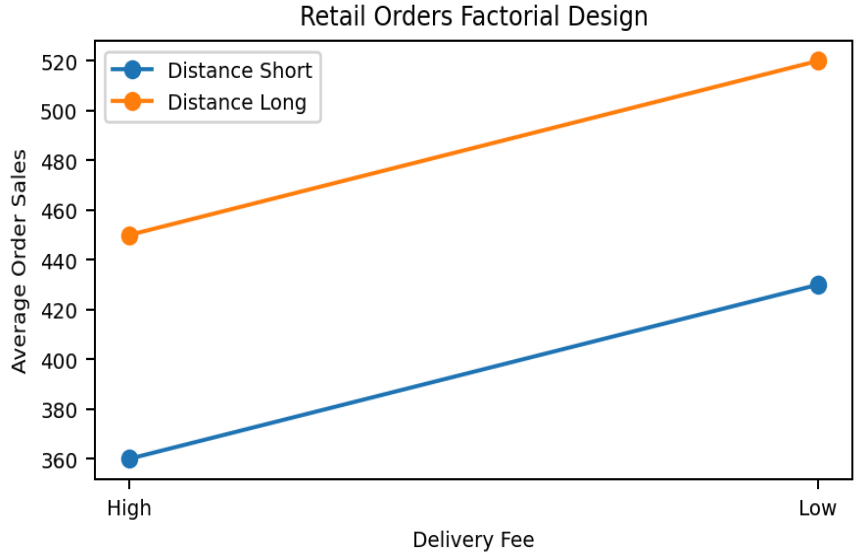

Distribution of Delivery Status
Count
400
300
200
100
0
Delivered
Cancelled
Delivery_Status


Retail Orders Delivery Locations
Latitude
Longitude
27.5
25.0
22.5
20.0
17.5
15.0
12.5
10.0
7.5
72
74
76
78
80
82
84
86
88



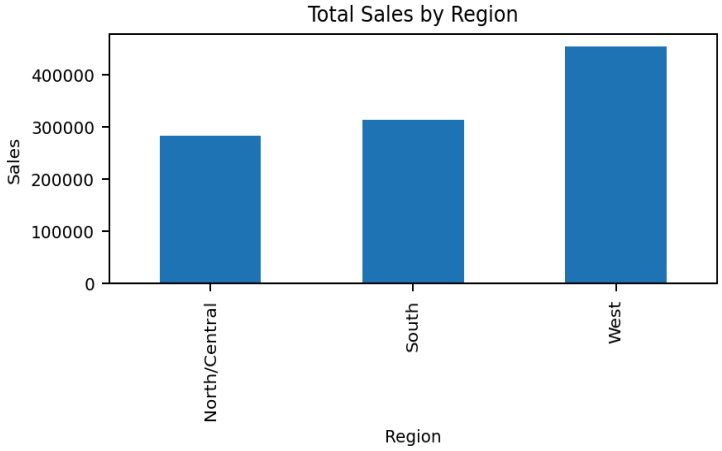
S

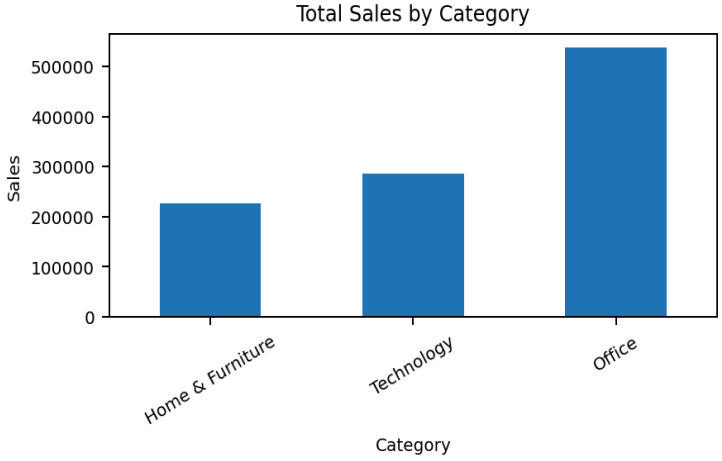


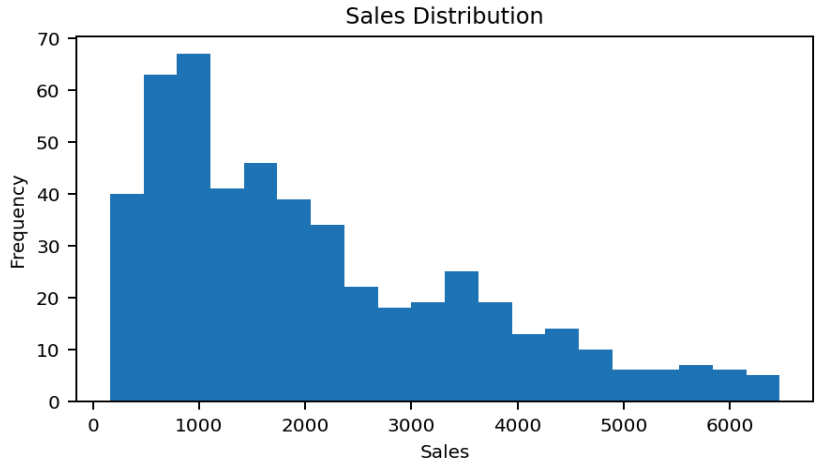


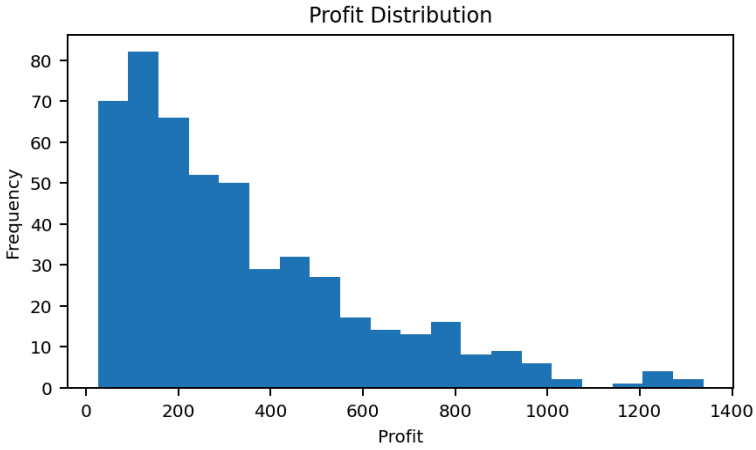
4] Additional Graphs:

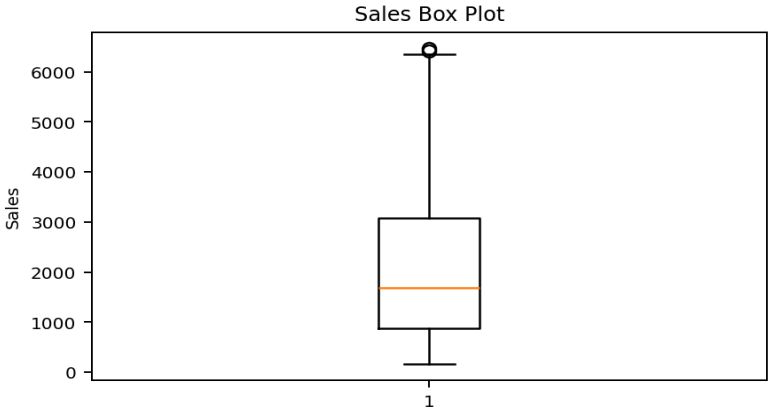
```
BASIC STATISTICS
       Quantity  Distance_KM    Sales   Profit
count    500.00       500.00   500.00   500.00
mean       3.95        12.98  2099.06   337.09
std        2.08         6.93  1489.37   260.18
min        1.00         1.84   160.78    26.19
25%        2.00         8.32   875.24   137.55
50%        4.00        11.72  1682.57   265.17
75%        6.00        16.29  3079.50   475.69
max        7.00        44.28  6469.83  1337.64
```

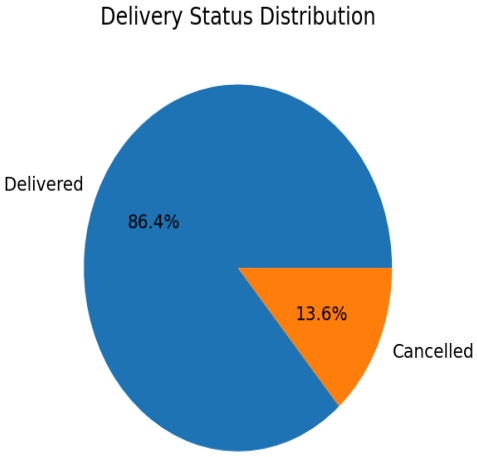


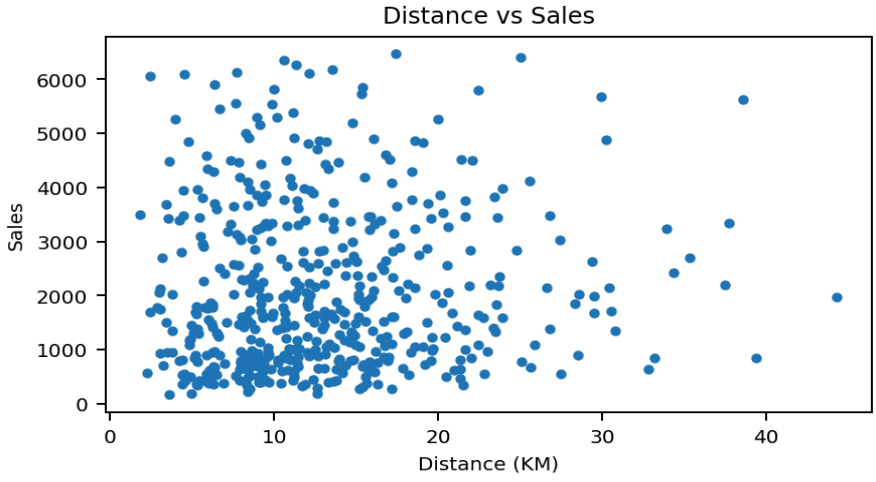




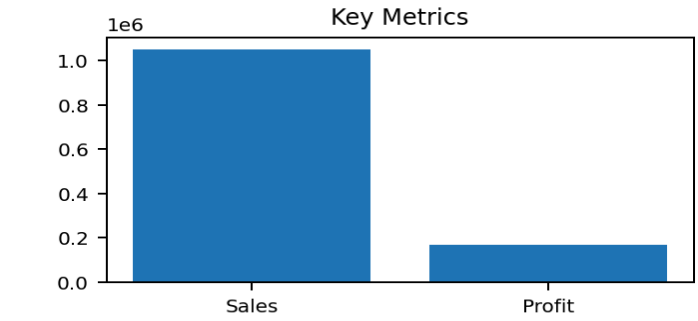

Profit Distribution
Frequency
Profit
0
10
20
30
40
50
60
70
80
0
200
400
600
800
1000
1200
1400

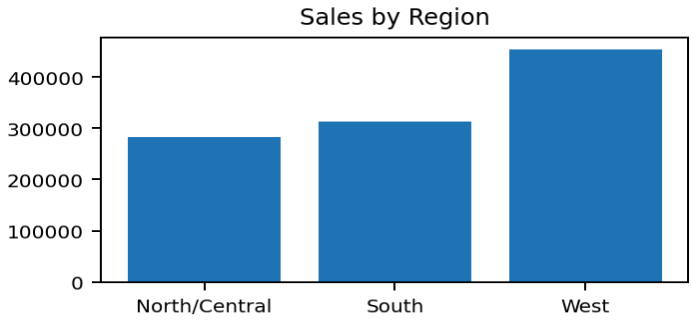

Sales Box Plot
Sales
0
1000
2000
3000
4000
5000
6000
1

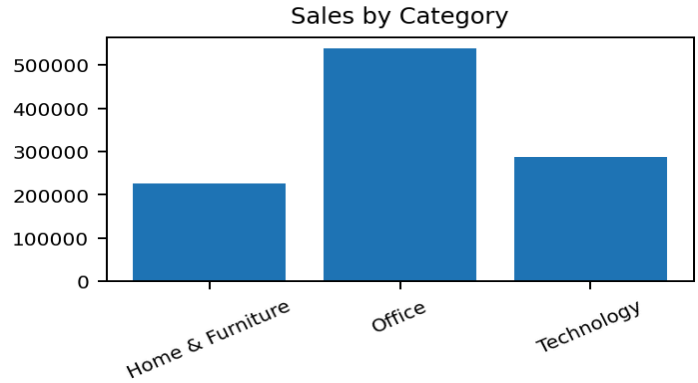

Delivery Status Distribution
Delivered
86.4%
13.6%
Cancelled

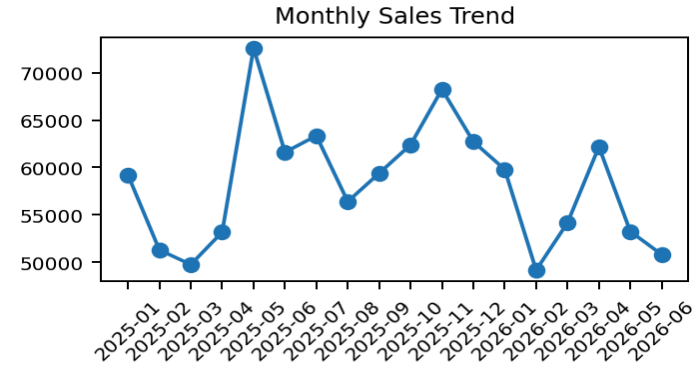

Distance vs Sales
Sales
Distance (KM)
0
1000
2000
3000
4000
5000
6000
0
10
20
30
40

5] Tableau Screenshots:









6] Decision Making Under Uncertainty:

```
Delivery_Status
Delivered    0.864
Cancelled    0.136
```

7] Optimization Output:

```
OPTIMIZATION SUCCESSFUL: True
Optimal Allocation: [0.  0. 37.5]
Maximum Estimated Profit: 1875.0
```

8] Simulation Output:

```
AVERAGE SIMULATED DELIVERED ORDERS: 432.04
MINIMUM: 405
MAXIMUM: 454
```